دليل تصميم الأنظمة

من المتطلبات إلى الموثوقية والأمان

عشرون قسما تعليميا

يتضمن هذا الملف الصور العشرين المرفقة بترتيبها الأصلي، مع صفحات عربية تشرح فكرة كل قسم ومحاوره ومثالا مبسطا عليه.

طريقة القراءة

اقرأ صفحة الشرح أولا، ثم افتح الصورة التالية وكبرها لمراجعة الجداول والمخططات الإنجليزية. الشرح تلخيص تفسيري، وليس ترجمة حرفية لكل سطر في الصور.

ملاحظة عن المرفقات

الصورة الأولى غلاف وفهرس. تليها صور الأقسام من 02 إلى 20. لا توجد صورة مستقلة للقسم 01، لذلك أضيف له شرح تمهيدي مع تمييزه بوضوح.

المحتويات

01 المتطلبات والنطاق 4

02 تقدير السعة 5

03 تصميم الواجهات البرمجية 7

04 التطبيق الموحد والخدمات المصغرة 9

05 موازنة الحمل 11

06 التخزين المؤقت 13

07 أساسيات قواعد البيانات 15

08 فهرسة قواعد البيانات 17

09 تكرار البيانات ونسخ القراءة 19

10 توزيع البيانات بين القواعد 21

11 الاتساق ونظرية كاب 23

12 طوابير الرسائل 25

13 النشر والاشتراك 27

14 تحديد معدل الطلبات 29

15 شبكة توزيع المحتوى وكاش الحافة 31

16 التخزين الموزع والملفات 33

17 توليد المعرفات الموزعة 35

18 تحمل الأعطال والموثوقية 37

19 قابلية الملاحظة والمراقبة 39

20 الأمان ومفاضلات التصميم 41

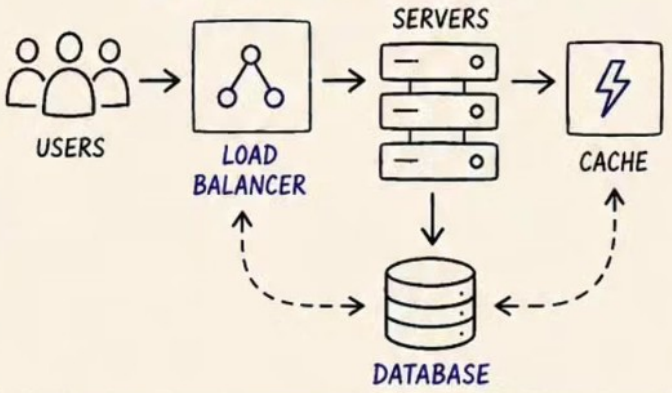
الغلاف الأصلي في الصفحة 3؛ كل صورة تفصيلية تلي شرح قسمها.

THE COMPLETE

SYSTEM DESIGN HANDBOOK

— 20 CONCEPTS. ONE COMPLETE GUIDE.

HOW A SYSTEM WORKS?



WHY THESE CONCEPTS MATTER?

- ✓ Build scalable & reliable systems
- ✓ Make better architectural decisions
- ✓ Crack system design interviews
- ✓ Be ready for real world problems





COMMENT FOR **FULL PDF GUIDE** IN DM

20 TOPICS INSIDE

01 Requirements & Scope
02 Capacity Estimation
03 API & Interface Design
04 Monolith vs Microservices
05 Load Balancing
06 Caching
07 Database Fundamentals
08 Database Indexing
09 Replication
10 Sharding
11 Consistency & CAP Theorem
12 Message Queues
13 Publish / Subscribe
14 Rate Limiting
15 CDN & Edge Caching
16 Distributed Storage
17 Distributed ID Generation
18 Fault Tolerance & Reliability
19 Observability
20 Security & Trade-offs

المتطلبات والنطاق

Requirements & Scope

ماذا يشرح هذا القسم؟

يبدأ تصميم النظام بتحديد ما يجب أن يفعله المنتج، ومن سيستخدمه، وما حدود العمل المطلوب. هذه صفحة تمهيدية مضافة؛ صورة القسم الأول غير موجودة ضمن المرفقات، رغم ورود عنوانه في فهرس الغلاف.

المحاور الأساسية

1. المتطلبات الوظيفية: العمليات التي ينفذها المستخدم، مثل التسجيل والبحث وإرسال الرسائل.
2. المتطلبات غير الوظيفية: زمن الاستجابة والتوافر والأمان وحجم الاستخدام المتوقع.
3. حدود النطاق: ما يدخل في النسخة الحالية، وما يؤجل إلى مراحل لاحقة.

مثال مبسط

في تطبيق اجتماعي، يمكن أن تبدأ بالمنشورات والإعجاب والرسائل، ثم تحدد الحجم المتوقع ومستوى الخدمة المطلوب قبل اختيار المكونات.

الفكرة التي ينبغي تذكرها

حدد المشكلة وحدودها أولا، ثم اختر الحل الذي يلائمها.

شرح تمهيدي مضاف؛ لا توجد صورة أصلية لهذا القسم ضمن المرفقات.

تقدير السعة

Capacity Estimation

ماذا يشرح هذا القسم؟

تشرح الصورة تحويل توقعات الاستخدام إلى تقديرات للطلبات والتخزين وعرض النطاق. الهدف هو بناء تصور لحجم الموارد المطلوبة بدل اختيار الخوادم اعتمادا على التخمين.

المحاور الأساسية

1. التفرقة بين إجمالي المستخدمين والمستخدمين النشطين يوميا أو شهريا.
2. حساب الطلبات في الثانية، ومتوسط الحمل مقابل حمل الذروة، ونسبة القراءة إلى الكتابة.
3. تقدير حجم البيانات اليومي والسنوي، مع توثيق الافتراضات ووحدات القياس.

مثال مبسط

المثال في الصورة يبدأ بعشرة ملايين مستخدم، ويفترض نشاط عشرين بالمئة يوميا، ثم يقدر عدد الطلبات وحجم تخزين الروابط.

الفكرة التي ينبغي تذكرها

التقدير تقريبي، ويصبح أكثر فائدة عندما تكون افتراضاته واضحة وقابلة للتعديل.

02

CAPACITY ESTIMATION

SYSTEM DESIGN DECODED

Why do we do it?

To estimate the scale of our system in terms of users, traffic, storage and bandwidth so that we can choose the right architecture and resources.

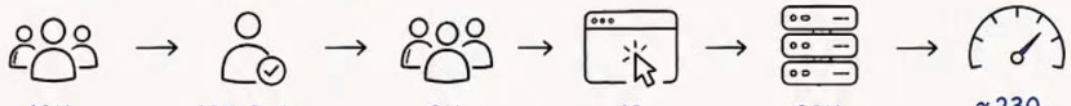
Golden Rule

Estimate, don't guess.
Be practical, not perfect.
Always state your assumptions.

What we estimate?

- ✓ DAU / MAU
- ✓ Requests per second (RPS)
- ✓ Peak traffic
- ✓ Read / Write ratio
- ✓ Storage estimation
- ✓ Bandwidth estimation

Step-by-step Estimation (Example)

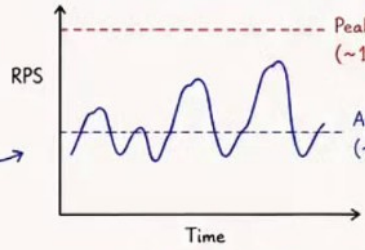


Step	Calculation	Example (URL Shortener)	Notes
1. Total Users	Given	10,000,000 users	From product goals / business data
2. Daily Active %	Total Users × % DAU	10,000,000 × 20% = 2,000,000	DAU usually 10% – 30% of MAU
3. Requests / User / Day	Given / Assumed	10 requests	Includes all actions per user
4. Requests / Day	DAU × Requests / User	2,000,000 × 10 = 20,000,000	Total requests in a day
5. Avg RPS	Requests / Day ÷ 86400	20,000,000 ÷ 86400 ≈ 231 RPS	86400 sec in a day
6. Peak RPS	Avg RPS × Peak Factor	231 × 5 ≈ 1155 RPS	Peak Factor typically 3x – 10x

Peak RPS ≠ Average RPS

Traffic is not constant!

It has peaks and valleys.



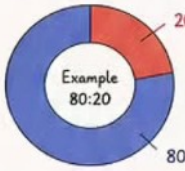
Plan for peak, not average. Systems fail during peaks, not averages!

Choosing Peak Factor

System Type	Peak Factor (x)
Internal / Admin tools	2x – 3x
Normal Consumer App	3x – 5x
Consumer Internet Scale	5x – 10x
Unpredictable / Viral	10x+

Read / Write Ratio

Helps in deciding DB, cache, replication and storage.



How to estimate?

- ✓ Read heavy → Use caching, replication
- ✓ Write heavy → Focus on DB write capacity, queue, sharding
- ✓ Balanced → Plan both carefully

Storage Estimation (Example)

Assume we store:
Short URL + Original URL + Metadata

Data	Size (approx)	Notes
Short code	20 bytes	e.g. aBcDe123
Original URL	200 bytes	Average URL length
Metadata	100 bytes	User id, createdAt, etc.
Total per record ≈ 320 bytes (0.32 KB)		

Assume 20M new URLs per day

Daily Storage = 20M × 0.32 KB
= 6.4 GB / day

Monthly Storage ≈ 6.4 × 30 = 192 GB

Yearly Storage ≈ 192 × 12 = 2.3 TB

Add 2x – 3x overhead for indexes, replication, backups, logs, etc.

Bandwidth Estimation (Example)

Assume average response size = 1 KB

20M requests/day × 1 KB
= 20,000,000 KB / day
≈ 20 GB / day

If we serve images/ads/files, bandwidth can be much higher.

Other Important Assumptions

- ✓ New users per day / month
- ✓ User growth rate (e.g., 20% MoM)
- ✓ Average session length

Things to keep in mind

- ☑ Estimates are direction, not exact numbers.
- ☑ Always validate with real metrics after launch.

Summary (Example)

Avg RPS	~ 231
Peak RPS	~ 1155
Read / Write Ratio	80 : 20

تصميم الواجهات البرمجية

API & Interface Design

ماذا يشرح هذا القسم؟

توضح الصورة الواجهة البرمجية بوصفها عقدا بين العميل والخدمة: كيف يرسل العميل الطلب، وكيف تصف الخدمة النتيجة أو الخطأ. التصميم الواضح يساعد الفرق والمكونات على التواصل.

المحاور الأساسية

1. تصميم الموارد والمسارات، واختيار طرق الطلب المناسبة مثل القراءة والإنشاء والتحديث والحذف.
2. أكواد الحالة والترويسات وشكل الطلب والاستجابة والمصادقة.
3. ترقيم الصفحات وإصدارات الواجهة للحفاظ على قابلية الاستخدام مع تطور المنتج.

مثال مبسط

يظهر مثال لطلب بيانات مستخدم، ثم استجابة تحتوي بياناته؛ كما تشرح الصورة الفرق بين نجاح الطلب وأخطاء العميل وأخطاء الخادم.

الفكرة التي ينبغي تذكرها

اجعل العقد واضحا ومتسقا، ولا تربط العميل بتفاصيل التنفيذ الداخلية.

03

API & INTERFACE DESIGN

SYSTEM DESIGN DECODED

Why APIs?

APIs define a contract between systems. They allow components to communicate in a clean, independent and scalable way.

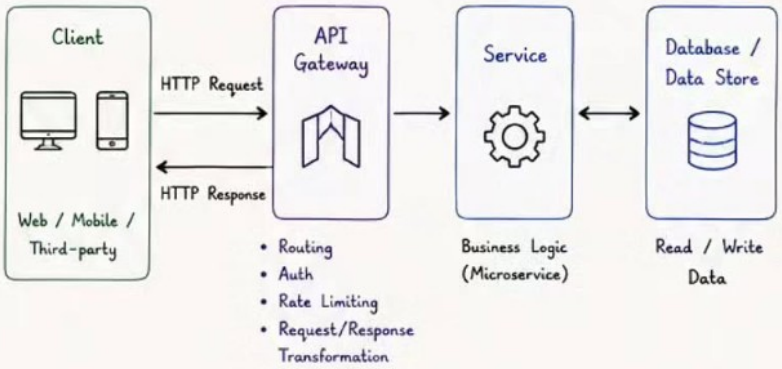
Golden Rule

Design APIs for simplicity, usability, consistency and scalability.

Key Concepts

- ✓ REST APIs
- ✓ Request / Response
- ✓ Resources & endpoints
- ✓ HTTP methods
- ✓ Status codes
- ✓ Pagination
- ✓ API versioning

How components communicate



Request / Response Example

Request (GET /users/123)

```
GET /users/123 HTTP/1.1
Host: api.example.com
Authorication: Bearer <token>
Accept: application/json
```

Response (200 OK)

```
HTTP/1.1 200 OK
Content-Type: application/json
{
 "id": 123,
 "name": "Nikhil",
 "email": "nikhil@example.com"
}
```

- Request line — HTTP Method, Endpoint, HTTP Version
- Headers — Metadata about the request
- Status line — Status Code, Message
- Headers — Metadata about the response
- Body — Actual data

REST API Basics

- ✓ Uses HTTP methods to perform actions on resources
- ✓ Resources are nouns, not verbs
- ✓ Stateless: each request contains all information needed
- ✓ Uniform interface makes APIs predictable and easy to use

Common Status Codes

Range	Code	Meaning	Use Case
2xx Success	200	OK	Request successful
	201	Created	Resource created
	204	No Content	Success, no content to return
3xx Redirection	301	Moved Permanently	Resource moved permanently
	304	Not Modified	Used with caching
4xx Client Error	400	Bad Request	Invalid request
	401	Unauthorized	Authentication required
	403	Forbidden	No permission
	404	Not Found	Resource not found
	429	Too Many Requests	Rate limit exceeded
5xx Server Error	500	Internal Server Error	Something broke
	503	Service Unavailable	Server temporarily down

HTTP Methods

Method	Purpose	Idempotent	Safe
GET	Retrieve a resource	Yes	Yes
POST	Create a new resource	No	No
PUT	Replace / Update a resource	Yes	No
PATCH	Partial update of a resource	No	No
DELETE	Delete a resource	Yes	No

Idempotent: Multiple same requests have the same effect as one request.

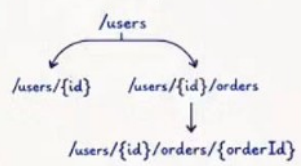
Safe: Does not modify data on the server.

Resources & Endpoints

Design around resources. Use nouns.

Good (Use nouns)	Bad (Avoid verbs)
/users	/getUsers
/users/123	/getUserById
/users/123/orders	/createOrder
/products	/deleteProduct
/products/456/reviews	/updateUser

Use hierarchical structure to show relationship.



Pagination

Limit the number of items returned in one response.

```
GET /users?page=2&limit=20&sort=createdAt:desc
```

Common Parameters:

- ✓ page → page number (starts from 1)
- ✓ limit → items per page
- ✓ sort → sort field and order
- ✓ Filters → e.g. ?role=admin, ?status=active

Response includes:

- data → items
- total → total items
- page, limit
- totalPages
- hasNext, hasPrev

API Versioning

Why? To introduce changes without breaking existing clients.

Common Approaches

1. URI Versioning	2. Header Versioning	3. Query Parameter	4. Media Type Versioning

Best Practice

Choose one and be

التطبيق الموحد والخدمات المصغرة

Monolith vs Microservices

ماذا يشرح هذا القسم؟

تقارن الصورة بين تطبيق موحد تنشر مكوناته معا، وخدمات مستقلة يمكن تطويرها ونشرها بصورة منفصلة. الاختيار يرتبط بحجم الفريق وتعقيد المجال والحاجة إلى الاستقلال.

المحاور الأساسية

1. التطبيق الموحد يسهل البداية والتشغيل، وقد يكون منظما إلى وحدات داخلية.
2. الخدمات المصغرة تمنح استقلالا أكبر، لكنها تضيف اتصالات شبكية وتنسيقا وتشغيلا أكثر تعقيدا.
3. التطبيق الموحد المعياري مرحلة ممكنة عندما تحتاج حدودا واضحة دون توزيع مبكر.

مثال مبسط

تعرض الصورة وظائف مثل المستخدمين والطلبات والدفع داخل تطبيق واحد، ثم تعرضها بوصفها خدمات منفصلة خلف بوابة.

الفكرة التي ينبغي تذكرها

ابدأ بالبساطة، وافصل الخدمات عندما توجد حاجة واضحة تدعم كلفة الفصل.

04

MONOLITH vs MICROSERVICES

SYSTEM DESIGN DECODED

Why this matters?

The right architecture depends on your team, scale, complexity and speed of change – not just popularity.

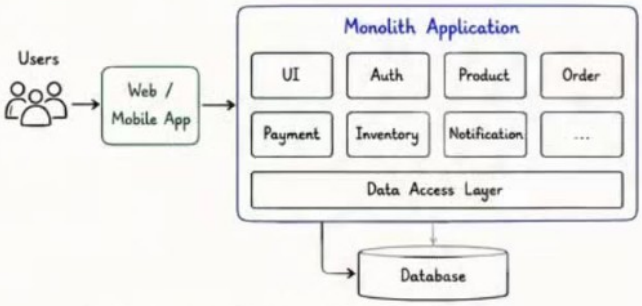
Golden Rule

Start simple.
Evolve only when needed.
Architecture should solve problems, not create new ones.

Key Concepts

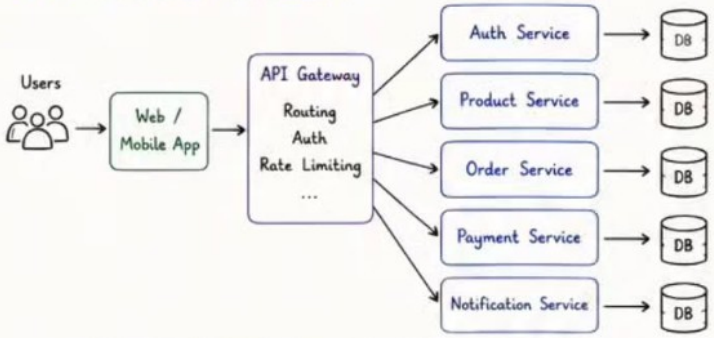
- ✓ Monolith
- ✓ Modular Monolith
- ✓ Microservices
- ✓ Service Boundaries
- ✓ Pros / Cons
- ✓ When to choose what

Monolith Architecture



- ✓ Everything is in a single deployable unit.
- ✓ One codebase, one database.

Microservices Architecture



- ✓ Services are small, independent and loosely coupled.
- ✓ Each service owns its data.

Monolith – Pros & Cons

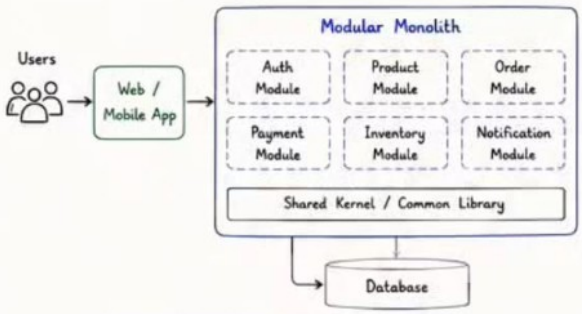
Pros (Good when...)	Cons (Watch out when...)
✓ Simple to develop and deploy	✗ Harder to scale (all or nothing)
✓ Easy testing (single codebase)	✗ Slower deployments as app grows
✓ Transactions are easy (single DB)	✗ Tech stack change is difficult
✓ Lower operational overhead	✗ High coupling – changes affect many
✓ Good for small teams & products	✗ One bug can bring down whole app
✓ Faster time to market initially	✗ Not ideal for large, complex systems

Microservices – Pros & Cons

Pros (Good when...)	Cons (Watch out when...)
✓ Independent deployment	✗ Higher operational complexity
✓ Scale services independently	✗ Network latency & failures
✓ Technology flexibility	✗ Distributed data / transactions are hard
✓ Fault isolation (one service down ≠ whole system down)	✗ More DevOps & monitoring
✓ Teams can own services	✗ Cost can increase
✓ Better for large, complex systems	✗ Not needed for small systems

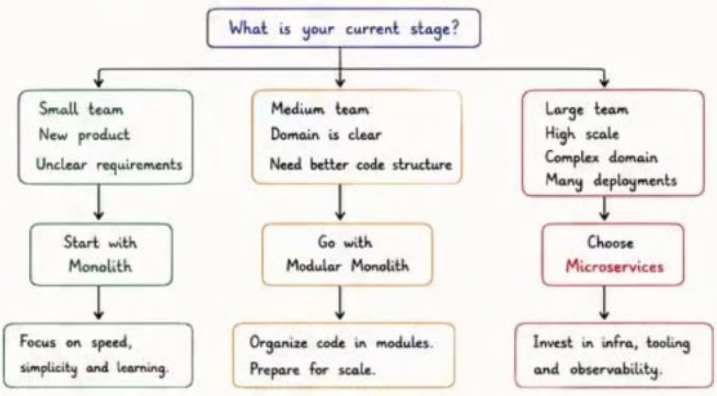
Modular Monolith

A middle ground. Single deployable, but code is well-structured into modules with clear boundaries.



- ✓ Better maintainability than monolith.
- ✓ Easier to move to microservices in future.

When to choose what?



☆ You can always evolve from left → right.
Don't start with microservices unless you truly need them.

Quick Comparison

Aspect	Monolith	Modular Monolith	Microservices
Structure	Single codebase, single deployable	Single deployable, modular code	Multiple services, independent deployables
Database	Single DB	Single DB	Per-service DB
Deployment	One unit	One unit	Many independent units
Scalability	Limited (vertical scaling)	Better (still limited)	High (scale per service)

موازنة الحمل

Load Balancing

ماذا يشرح هذا القسم؟

تشرح الصورة استقبال الطلبات في نقطة دخول وتوزيعها على خوادم التطبيق السليمة. يساعد ذلك على استثمار عدة خوادم وتجنب توجيه العمل إلى خادم متعطل.

المحاور الأساسية

1. خوارزميات التوزيع مثل التناوب وأقل عدد من الاتصالات والتوزيع الموزون.
2. فحوص الصحة لاكتشاف الخادم غير المتاح واستبعاده من التوجيه.
3. الفرق بين الموازنة على مستوى النقل والموازنة على مستوى التطبيق، وتأثير الجلسات الثابتة.

مثال مبسط

عند تشغيل ثلاثة خوادم للتطبيق، يوزع الموازن الطلبات بينها بحسب السياسة المختارة وحالتها الصحية.

الفكرة التي ينبغي تذكرها

توزيع الحمل يحتاج سياسة مناسبة وفحوص صحة، مع الانتباه إلى توافر الموازن نفسه.

05

LOAD BALANCING

SYSTEM DESIGN DECODED

Why do we need it?

A single server can become a bottleneck. It can get overloaded and crash. Load balancing distributes incoming traffic across multiple servers to improve performance, availability and reliability.

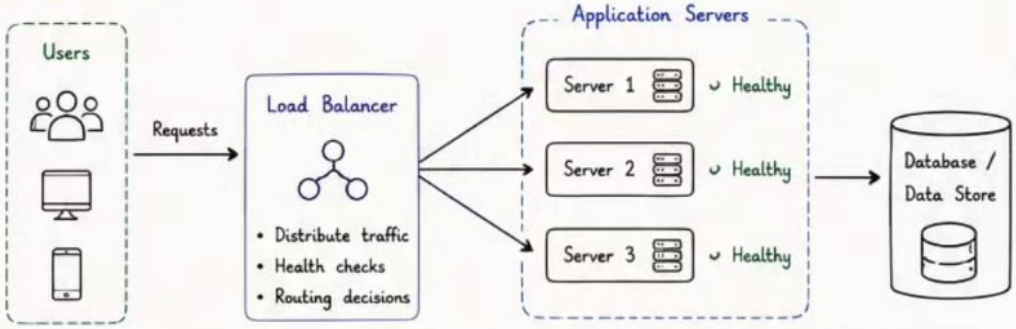
Golden Rule

Distribute traffic.
Avoid overload.
No single point of failure.

Key Concepts

- ✓ Load Balancer Role
- ✓ Health Checks
- ✓ Routing Algorithms
- ✓ Round Robin
- ✓ Least Connections
- ✓ Weighted Routing
- ✓ L4 vs L7 Load Balancer

How it works



If a server is down, traffic is automatically sent to healthy servers.

Health Checks

LB periodically checks servers. If a server fails to respond, it is marked unhealthy and removed from rotation.

- Check Interval: 5s
- Timeout: 2s
- Unhealthy Threshold: 3
- Healthy Threshold: 2

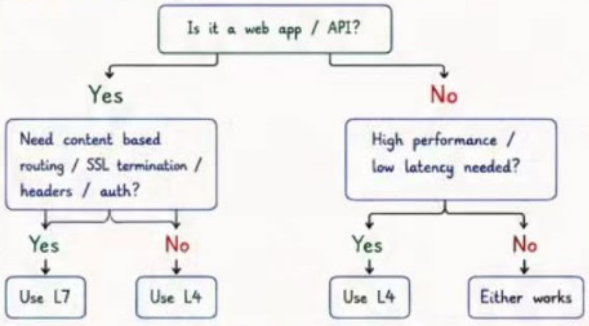
Routing / Load Balancing Algorithms

Algorithm	How it works	Best for	Pros	Cons
Round Robin ① → ② → ③ → ① ...	Requests are distributed sequentially to each server in order.	Servers with similar capacity and low complexity	✓ Simple ✓ Fair distribution ✓ No configuration needed	× Ignores server health × Not ideal if servers have different capacities
Least Connections 7 2 5	Request is sent to the server with the least number of active connections.	Long-lived connections (e.g., WebSocket, Streaming)	✓ Better for variable load ✓ Considers current load	× Slightly more complex × Not perfect for short requests
Weighted Round Robin ① ① ② ② ② ③ ③ ① ...	Servers have weights. Requests are distributed based on weight.	Servers with different capacities	✓ Utilizes server capacity efficiently ✓ Simple to implement	× Weights need tuning × Doesn't consider current load
IP Hash (Sticky Sessions)	Requests from the same IP always go to the same server.	Session persistence without external store	✓ Simple session affinity ✓ Good for legacy apps	× Uneven distribution if IP range is skewed × Server changes cause rebalancing issues

L4 vs L7 Load Balancing

Aspect	L4 (Transport Layer)	L7 (Application Layer)
Works at	Layer 4 (TCP/UDP)	Layer 7 (HTTP/HTTPS)
Routing based on	IP, Port	URL, Host, Headers, Cookies, Path
Protocol awareness	No	Yes
Performance	Higher (faster)	Slightly lower (more processing)
Use cases	High throughput, low latency, non-HTTP apps	Web apps, APIs, content-based routing, auth, rate limiting
Examples	Nginx (stream), AWS NLB, HAProxy (TCP)	Nginx (HTTP), AWS ALB, HAProxy (HTTP)

When to use what?



Benefits

Remember

التخزين المؤقت

Caching

ماذا يشرح هذا القسم؟

تشرح الصورة الاحتفاظ بنتائج كثيرة الاستخدام في طبقة أسرع، لتقليل العمل المتكرر على قاعدة البيانات. وتميز بين العثور على البيانات في الكاش وعدم وجودها أو انتهاء صلاحيتها.

المحاور الأساسية

1. سياسات القراءة والكتابة: القراءة عبر الكاش، والتحميل عند الحاجة، والكتابة المباشرة أو المؤجلة.
2. مدة الصلاحية وإبطال النسخة عند تغير البيانات، بدل الاعتماد على بيانات قديمة بلا ضابط.
3. التفرقة بين نجاح القراءة من الكاش، وفقدانها، والحاجة إلى الرجوع إلى المصدر.

مثال مبسط

عند عرض ملف شخصي متكرر، يمكن تحميله من قاعدة البيانات مرة ثم خدمة قراءات لاحقة من الكاش ما دامت النسخة صالحة.

الفكرة التي ينبغي تذكرها

سرعة الكاش مرتبطة أيضا بصحة البيانات وخطة تحديثها.

06

CACHING

SYSTEM DESIGN DECODED

Why do we cache?

Database is slow & expensive.
Cache stores frequently accessed data in memory so that we can respond faster and reduce load on database.

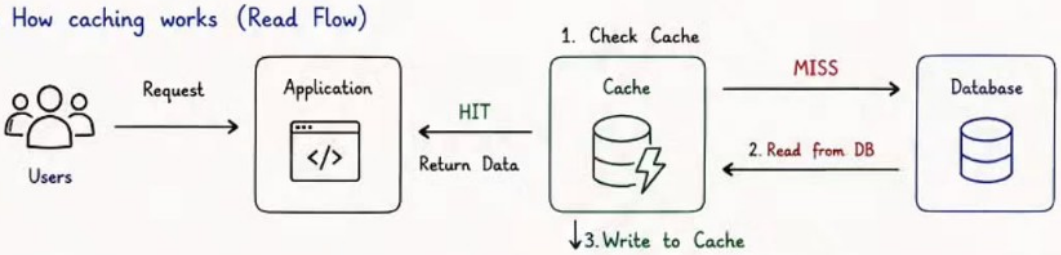
Golden Rule

Cache what is expensive.
Cache what is frequently used.
Keep cache correct.

Key Concepts

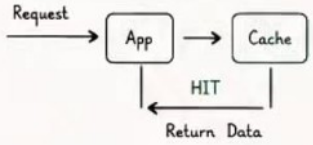
- ✓ Cache Basics
- ✓ Cache-Aside
- ✓ Read-Through
- ✓ Write-Through
- ✓ Write-Back
- ✓ TTL (Time To Live)
- ✓ Cache Invalidation
- ✓ Cache Hit / Miss

How caching works (Read Flow)



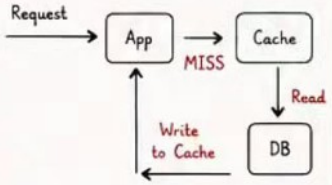
HIT = Data found in cache
MISS = Data not in cache

CASE 1: CACHE HIT



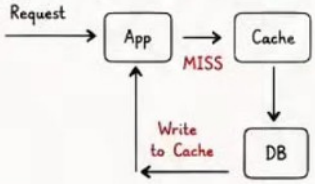
- ✓ Fast response
- ✓ DB not called

CASE 2: CACHE MISS



- ✓ Slower than hit
- ✓ Cache gets updated

CASE 3: CACHE EXPIRED (TTL expired)



- ✓ Treated as miss
- ✓ Fresh data loaded

TTL (Time To Live)

Every cache entry has an expiry time. After TTL, the entry is considered stale and removed.

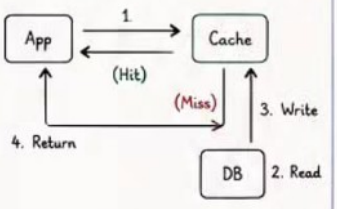
Cache Hit / Miss

Hit Ratio = Hits / (Hits + Misses)

Caching Patterns

1) Cache-Aside (Lazy Loading)

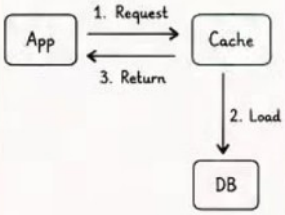
1. App checks cache
2. If miss, read from DB
3. Write to cache
4. Return to user



Most popular & widely used. App controls caching.

2) Read-Through

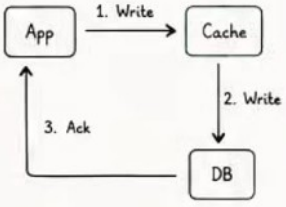
1. App requests data
2. Cache checks & fetches
3. Cache returns data



Cache handles DB loading. Simpler for the app.

3) Write-Through

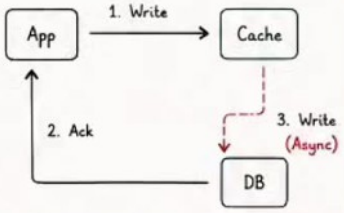
1. App writes data
2. Write to cache
3. Write to DB (synchronously)



Data always written to cache and DB together. Consistent but slower writes.

4) Write-Back (Write-Behind)

1. App writes data
2. Write to cache
3. Write to DB (asynchronously)



Fast writes. DB updated later. Risk of data loss if crash before sync.

Cache Invalidation

Keeping cache fresh is the hardest part.

When to invalidate?

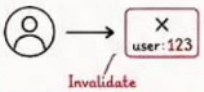
- ✓ When data is updated / deleted
- ✓ When TTL expires
- ✓ When related data changes
- ✓ When business rules demand

How to invalidate?

- Delete the key
- Update the key
- Expire the key (set TTL = 0)
- Version the key

Example

User updates profile.
→ Invalidate: user:123
→ Next read will fetch fresh data.



Good Candidates

- ✓ Read-heavy data
- ✓ Frequent & repetitive data
- ✓ Reference / master data
- ✓ Expensive DB queries
- ✓ Session / tokens
- ✓ Rendered pages / responses
- ✓ Static or slow-changing data

Avoid Caching

- × Highly dynamic data
- × User-specific sensitive data
- × Data needing strong consistency
- × Very large datasets
- × One-time or rare data
- × Data with complex invalidation

Quick Comparison

أساسيات قواعد البيانات

Database Fundamentals

ماذا يشرح هذا القسم؟

تعرض الصورة نماذج قواعد البيانات ووظائفها الأساسية، وتقارن التخزين العلائقي بالنماذج غير العلائقية. كما تقدم المخططات والفهارس والمعاملات بوصفها عناصر تؤثر في تصميم البيانات.

المحاور الأساسية

1. النماذج العلائقية تعتمد الجداول والعلاقات؛ وتشمل النماذج الأخرى المستندات والمفاتيح والقيم والرسوم البيانية.

2. المخطط يصف بنية البيانات، والفهرس يساعد على الوصول إليها، والمعاملة تجمع عمليات مترابطة.

3. خصائص المعاملات تشمل الذرية والاتساق والعزل والديمومة.

مثال مبسط

عملية تحويل رصيد تحتاج معالجة مترابطة لخصم المبلغ وإضافته؛ وتستخدم الصورة معاملات ناجحة وأخرى تتراجع عنها لتوضيح الفكرة.

الفكرة التي ينبغي تذكرها

اختر نموذج البيانات وفق العلاقات والاستعلامات والضمانات التي تحتاجها.

07

DATABASE FUNDAMENTALS

SYSTEM DESIGN DECODED

Why Databases?

Databases store data persistently, organize it efficiently and allow reliable access even at scale.

Golden Rule

Choose the right database for your problem, not the most popular one.

Key Concepts

- ✓ Relational DB
- ✓ NoSQL DB
- ✓ Schema
- ✓ Indexes
- ✓ Transactions
- ✓ ACID
- ✓ When SQL / NoSQL makes sense

Types of Databases

Relational Database (SQL)

UserID	Name	Email	CreatedAt
1	Nikhil	nikhil@example.com	2024-05-01
2	Priya	priya@example.com	2024-05-01
...	...	...	...

- Data in tables (rows & columns)
- Fixed / predefined schema
- Strong consistency
- Powerful querying with JOINs
- Examples: MySQL, PostgreSQL, Oracle

NoSQL Database

Document (MongoDB)

```
{
  "_id": 1,
  "name": "Nikhil",
  "email": "nikhil@example.com",
  "age": 27,
  "skills": ["js", "sys design"]
}
```

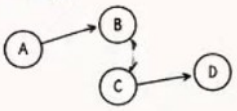
Column Family (Cassandra)

RowKey	Col1	Col2	Col3
user1	v1	v2	v3
user2	v1	v2	v3

Key-Value (Redis)

Key	Value
user:1	(JSON)
session:abc	token123

Graph (Neo4j)



- Flexible / dynamic schema
- Built for scale-out (horizontal scaling)
- High availability & partition tolerance
- Examples: MongoDB, Cassandra, Redis, Neo4j

SQL vs NoSQL

Aspect	Relational DB (SQL)	NoSQL DB
Data Model	Tables (rows & columns)	Key-Value / Document / Column / Graph
Schema	Fixed schema	Flexible schema (schema-less)
Query Language	SQL	Key-based / Document-based / Query APIs
Relationships	Strong (JOINs, Foreign Keys)	Usually denormalized / Application level
Transactions	Multi-row ACID transactions	Limited / Optional (varies by DB)
Scalability	Scale-up (vertical) mostly	Scale-out (horizontal) easily
Consistency	Strong consistency	Eventual consistency (mostly)
Use Cases	Financial systems, ERP, CRM, Reporting, Complex queries	Real-time apps, Caching, Logs, IoT, Content, Social, Big Data
Examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, DynamoDB, Neo4j

Schema

Schema defines how data is structured. Good schema = Less redundancy, better performance.

SQL: Fixed schema

Users

id	name	email	age
1	Nikhil	nikhil@...	27

NoSQL: Flexible schema

User A

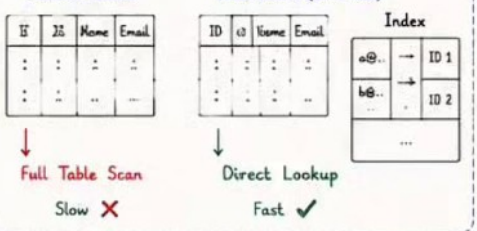
```
{
  id: 1,
  name: "Nikhil",
  age: 27
}
```

User B

```
{
  id: 2,
  name: "Priya",
  city: "Pune"
}
```

Indexes

Indexes are like the index of a book. They help database find data faster.



Good to index:

- ✓ Columns used in WHERE, JOIN, ORDER BY
- ✓ Unique / high cardinality columns

Avoid indexing:

- ✗ Very small tables
- ✗ Columns with low cardinality (e.g. gender)

Transactions

A transaction is a unit of work. Either everything succeeds or nothing happens.

Example: Bank Transfer

Steps

1. Debit from A
2. Credit to B
3. Update balance
4. Send notification

Without Transaction

- Step 1 done ✓
- Step 2 failed ✗
- Inconsistent state ✗

BAD

With Transaction (ACID)

All or nothing. System remains consistent.

GOOD

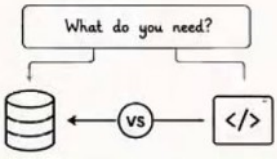
ACID Properties (Relational DBs)

A	Atomicity	All operations in a transaction succeed or none.
C	Consistency	Transaction brings DB from one valid state to another valid state.
I	Isolation	Transactions are isolated from each

When to use SQL vs NoSQL?

Go with SQL when:

- ✓ Structured data
- ✓ Complex queries / JOINs
- ✓ Strong consistency



Go with NoSQL when:

- ✓ Large scale, high volume
- ✓ Flexible / evolving schema
- ✓ High write throughput

فهرسة قواعد البيانات

Database Indexing

ماذا يشرح هذا القسم؟

توضح الصورة كيف يختصر الفهرس الوصول إلى الصف المطلوب مقارنة بفحص الصفوف بالتتابع. وتعرض الفهرس الشجري والفهارس المركبة وكلفة المحافظة عليها.

المحاور الأساسية

1. الفهرس المناسب يرتبط بالأعمدة وشروط البحث المستخدمة فعليا.
2. ترتيب الأعمدة مهم في الفهرس المركب، وقد يساعد الفهرس المغطي على تقليل الرجوع إلى الجدول.
3. الفهارس تستهلك مساحة وتزيد عمل الكتابة؛ لذلك كثرتها ليست هدفا بحد ذاتها.

مثال مبسط

إذا كان البحث المتكرر بالبريد الإلكتروني، فالفهرس على هذا الحقل قد يختصر الوصول إلى المستخدم بدل مسح الجدول كاملا.

الفكرة التي ينبغي تذكرها

ابن الفهارس بحسب الاستعلامات، ثم تحقق من خطة التنفيذ والأداء.

08

DATABASE INDEXING

SYSTEM DESIGN DECODED

Why Indexes?

Indexes help the database find data without scanning every row.
They act like the index of a book – pointing us directly to the right page.



→ Index — Directs you to the right page

Golden Rule

Indexes make reads FAST,
But every index has a cost on writes.
Use the right index, not many indexes.

Key Concepts

- ✓ What is an index?
- ✓ B-Tree intuition
- ✓ Composite index
- ✓ Query speed
- ✓ Write overhead
- ✓ When indexes hurt
- ✓ Index selectivity

1. Without Index (Full Table Scan)

Find users where email = 'nikhil@example.com'

id	name	email	age
1	Aman	aman@example.com	25
2	Priya	priya@example.com	28
3	Ravi	ravi@example.com	31
...	...	...	...
9999998	Neha	neha@example.com	22
9999999	Karan	karan@example.com	29
10000000	Nikhil	nikhil@example.com	27

Same query

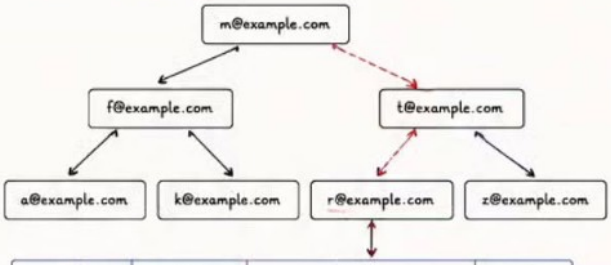
Huge difference in work done!

- Checks every single row one by one
- Slow for large tables
- Cost grows linearly with data size O(n)

2. With Index (Using Index)

Find users where email = 'nikhil@example.com'

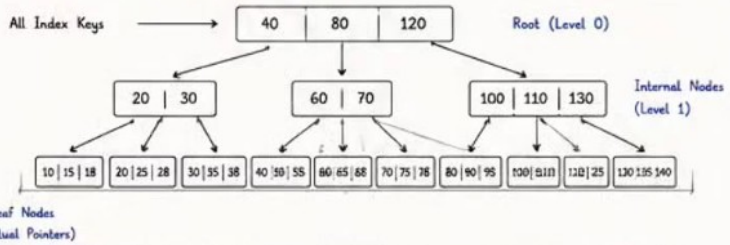
Index on email (B-Tree)



id	name	email	age
10000000	Nikhil	nikhil@example.com	27

- Database uses index to jump to the right location
- Reads only a few index pages + the actual data row
- Cost grows logarithmically O(log n)

3. B-Tree Intuition (How it works)



- Balanced tree
- Each node has multiple keys & pointers
- Height of tree is very small (log n)
- DB jumps down the tree to reach the key quickly

Example: 10 million rows
If each node has ~100 keys,
Height of tree ≈ 3 or 4
(Only 3–4 disk reads!)

4. Composite Index (Multi-Column Index)

Example: Index on (city, created_at)

Index Entries (city, created_at)

('Bangalore', 2023-01-01)	→	RowID 101
('Bangalore', 2023-01-02)	→	RowID 105
('Bangalore', 2023-01-03)	→	RowID 110
('Delhi', 2023-01-01)	→	RowID 201
('Delhi', 2023-01-02)	→	RowID 205
('Mumbai', 2023-01-01)	→	RowID 301
...		

Leftmost prefix rule:
Index (A, B, C) can be used for queries on (A), (A, B) or (A, B, C) but NOT on (B) or (C) alone.

✓ Good use
WHERE city = 'Bangalore'
ORDER BY created_at

✗ Not ideal
WHERE created_at = '2023-01-02'

5. Indexes Improve Query Speed

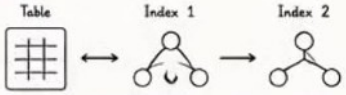
Data Size (rows)	Full Table Scan (Time Complexity)	Index Lookup (Time Complexity)
10K	~ 10,000 reads	~ 3–4 reads
100K	~ 100,000 reads	~ 3–4 reads
1M	~ 1,000,000 reads	~ 3–4 reads
10M	~ 10,000,000 reads	~ 3–4 reads

Indexes give huge improvement on large datasets & selective queries (queries that return a small number of rows).

6. Write Overhead (The Cost)

Every time you INSERT / UPDATE / DELETE:

- ✗ Index also needs to be updated
- ✗ More disk writes
- ✗ More locks / contention
- ✗ More storage used



- More indexes [ ] Slower writes

7. When Indexes Hurt

- ✗ Low selectivity columns (e.g., gender with only 'M' or 'F')
- ✗ Too many indexes on a table
- ✗ Columns used in frequent updates
- ✗ Small tables (full scan may be faster)
- ✗ Overhead > Benefit

- Not every column needs an index.
- Measure first, then add.

8. Index Types (Quick Look)

Index Type	Best For	Pros	Cons	Example
Primary Key	Unique rows	Fast lookups, uniqueness	Only one per table	id

Pro Tips

- ✓ Index foreign keys

تكرار البيانات ونسخ القراءة

Replication

ماذا يشرح هذا القسم؟

تشرح الصورة نسخ بيانات القاعدة الأساسية إلى نسخ أخرى، لتوزيع القراءات ودعم استمرارية الخدمة. كما تفرق بين التكرار المتزامن وغير المتزامن والتحويل إلى بديل.

المحاور الأساسية

1. الكتابة على الأساسية والقراءة من النسخ في النموذج المعروض.
2. التكرار غير المتزامن قد يسبب تأخر النسخ؛ والمتزامن يضيف انتظارا لتحقيق ضمانات أقوى.
3. التحويل إلى بديل يحتاج اكتشاف العطل واختيار نسخة مناسبة وإعادة توجيه الطلبات.

مثال مبسط

قد يحدث مستخدم ملفه على الأساسية، بينما تخدم طلبات العرض من نسخ القراءة؛ ويجب أخذ تأخر التحديث في الحسبان.

الفكرة التي ينبغي تذكرها

نسخ القراءة تساعد القراءة، لكن لا تعني تلقائيا توزيع الكتابة.

09

REPLICATION

SYSTEM DESIGN DECODED

• Why Replication?

Replication keeps a copy of data on multiple database servers so that we can improve availability, performance and fault tolerance.

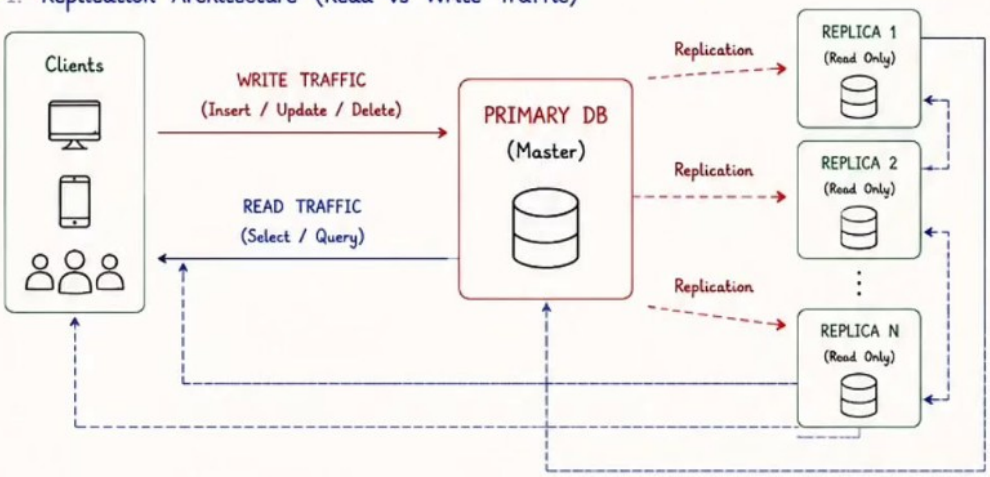
Golden Rule

Write to Primary.
Read from Replicas.
Be available, even if one fails.

• Key Concepts

- ✓ Primary / Replica
- ✓ Read Replicas
- ✓ Replication Flow
- ✓ Synchronous vs Asynchronous
- ✓ Read Scaling
- ✓ Failover

• 1. Replication Architecture (Read vs Write Traffic)

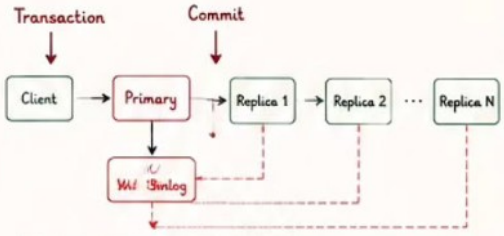


Red = Write Path (Only to Primary)
Blue = Read Path (From Replicas)

- All writes go to Primary DB.
- Primary replicates changes to all replicas.
- Clients read from replicas.
- If a replica is slow, remove it from rotation.
- Scale reads by adding more replicas.

• 2. Replication Flow (Step by Step)

1. Client writes to Primary DB.
2. Primary writes to its WAL (Write-Ahead Log).
3. Changes are sent to replicas.
4. Replicas apply the changes.
5. Replicas are now up-to-date.



Replication is usually async (replicas may be slightly behind).

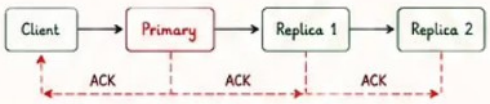
• 3. Primary vs Replica

	Primary (Master)	Replica (Slave)
Purpose	Accepts all writes	Serves read requests
Data	Up-to-date	May be slightly behind
Workload	Write heavy	Read heavy
Failure Impact	High (critical)	Low (reads can shift)
Scale	Usually 1	Many (0...N)

• 4. Synchronous vs Asynchronous Replication

Synchronous Replication

Write is successful when replicas acknowledge.



✓ Strong durability, no data loss (if majority alive)
✗ Higher latency, lower availability (if replica slow → write waits)

Asynchronous Replication (Most common)

Write is successful after commit on primary.



✓ Lower latency, higher availability
✗ Possible data loss (if primary crashes before replicas catch up)

Which to use?

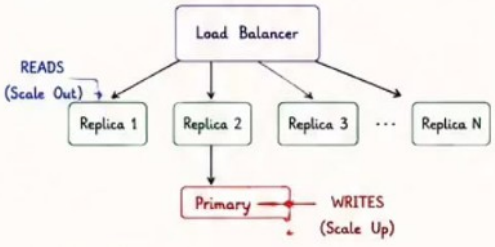
Synchronous → Banking, Payments, Strong consistency systems

Asynchronous → Most web apps, Social, E-commerce, Analytics

Most systems use async with good monitoring & backups.

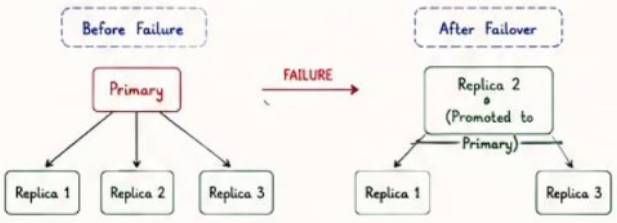
• 5. Read Scaling

Add more read replicas to handle more read traffic.



• 6. Failover (Automatic or Manual)

If primary fails, a replica is promoted to new primary.



- Detect failure (health check).
- Stop writes to old primary.
- Promote a replica.
- Update clients / routes.
- Re-estup replication from new primary to other replicas.

• 7. Best Practices

توزيع البيانات بين القواعد

Sharding

ماذا يشرح هذا القسم؟

تشرح الصورة تقسيم مجموعات من الصفوف بين قواعد مستقلة باستخدام مفتاح توزيع. الهدف هو توزيع حجم البيانات والحمل، مع الحفاظ على القدرة على توجيه الطلب إلى القسم الصحيح.

المحاور الأساسية

1. اختيار مفتاح التوزيع يؤثر في توازن الحمل وتجمع بيانات المستخدم أو العميل.
2. يمكن التوزيع بحسب النطاق أو التجزئة، وتعرض الصورة التجزئة المتسقة وإعادة التوزيع.
3. الأقسام الساخنة والاستعلامات الشاملة ونقل البيانات تزيد تعقيد التشغيل.

مثال مبسط

يمكن توجيه بيانات العميل إلى قسم معين اعتمادا على معرفه؛ أما تقرير يشمل كل العملاء فقد يحتاج جمع نتائج عدة أقسام.

الفكرة التي ينبغي تذكرها

اختر مفتاحا يناسب الوصول إلى البيانات، ولا تنتقل للتوزيع قبل ظهور حاجة حقيقية.

10

SHARDING

SYSTEM DESIGN DECODED

Why Sharding?

Replication helps scale reads.
But when data volume becomes huge, a single server can't store or handle it.
Sharding splits the data across multiple machines.

Replication = More copies
Sharding = Split the data

Golden Rule

Split big data sets horizontally across multiple machines.
Distribute smartly.
Keep it balanced.

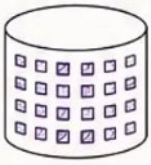
Key Concepts

- ✓ What is Sharding?
- ✓ Horizontal Partitioning
- ✓ Shard Key
- ✓ Consistent Hashing
- ✓ Hot Shards
- ✓ Resharding

1. What is Sharding?

Without Sharding

All data on one server

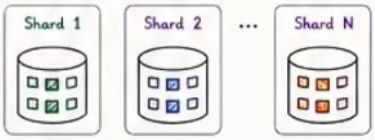


Limits:
- Storage limit
- High load
- Single point of failure

Sharding →

With Sharding

Data split across multiple servers



Benefits:
- Scale storage
- Scale writes
- Better performance
- Fault isolation

2. Horizontal Partitioning

Split rows across shards (not columns).

Example: Users Table

id	name	email	country	created_at
1	Aarti	aarti@mail.com	IN	2024-01-01
2	Bharat	bharat@mail.com	IN	2024-01-02
3	Charlie	charlie@mail.com	US	2024-01-03
...	...	...	...	...
10000001	Dev	dev@mail.com	UK	2024-05-01
10000002	Eva	eva@mail.com	UK	2024-05-02

Shard 1 (Range 1 – 10M)	Shard 2 (Range 10M+1 – 20M)	Shard N (Range ...)
1 ... 10000000	10000001 ... 20000000	20000001 ... N

3. Shard Key

Column (or set of columns) used to decide which shard a row goes to.

Example Table: Users

id	name	email	country
1	Aarti	aarti@mail.com	IN
2	Bharat	bharat@mail.com	IN
3	Charlie	charlie@mail.com	US
4	David	david@mail.com	US
5	Eve	eve@mail.com	UK
6	Frank	frank@mail.com	AU

Shard Key Options

user_id
(Good for even distribution if IDs are random)

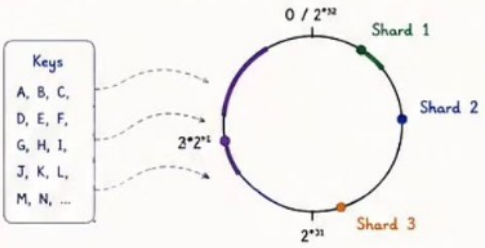
country
(Good if traffic is per country)

created_at (date)
(Good for time-series / archival)

Avoid low-cardinality keys
(e.g., gender, status)

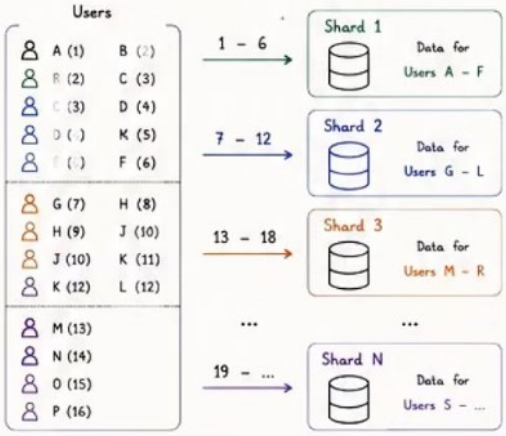
4. Consistent Hashing (How it helps)

Maps shard key to a shard using a hash ring.
When nodes change, only a small portion of data moves.



Add/Remove a shard → only nearby keys move, not all.

5. Sharding Example (User_id based)



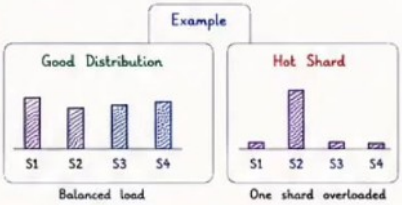
6. Hot Shards (Problem)

Uneven data or traffic causes some shards to become hotspots.

Causes
- Poor shard key
- Skewed data
- Trending entities

Symptoms
- High CPU / IOPS
- High latency
- Uneven storage

Example



How to fix?
- ✓ Choose a better shard key
- ✓ Add more shards
- ✓ Move / Split hot shard
- ✓ Use consistent hashing

7. Resharding (When & How)

When data grows or distribution becomes uneven, we reshard.

Triggers
- Shard size too big
- Uneven load
- New regions / nodes

Resharding Steps (High Level)

1. Plan new shard setup
2. Add new shards
3. Rebalance data (live migration)
4. Update routing / metadata
5. Decommission old shards

Goal: Move minimal data, keep system available.

8. Good Shard Key vs Bad Shard Key

Good Shard Key

✓ High cardinality (many unique values)

Why good?
Even data distribution

Bad Shard Key

✗ Low cardinality (few unique values)

Examples
gender (M/F)

الاتساق ونظرية كاب

Consistency & CAP Theorem

ماذا يشرح هذا القسم؟

توضح الصورة ما يحدث عندما يمنع انقسام الشبكة أجزاء النظام الموزع من التواصل. عندها توجد مفاضلة بين اتساق قوي للقراءات واستمرار الإجابة على الطلبات في الأجزاء المنفصلة.

المحاور الأساسية

1. الاتساق في سياق النظرية يتعلق برؤية القراءة لقيمة حديثة وفق ترتيب موحد للعمليات.
2. التوافر في سياق النظرية يتعلق بالحصول على استجابة للطلب؛ وهو أضيق من الاستخدام العام لكلمة التوافر.
3. أثناء انقسام الشبكة، قد ترفض جهة بعض الطلبات لحماية الاتساق، أو تستجيب مع قبول اختلاف مؤقت.

مثال مبسط

تعرض الصورة عقدتين تنقطع الصلة بينهما: إما تقييد العمل لتجنب اختلاف البيانات، أو السماح بالعمل ثم معالجة الاختلاف لاحقا.

الفكرة التي ينبغي تذكرها

تظهر المفاضلة أثناء انقسام الشبكة، ويجب ربط الاختيار بطبيعة البيانات والعمليات.

11

CONSISTENCY & CAP THEOREM

SYSTEM DESIGN DECODED

- **Why can't we have everything perfect?**

In a distributed system, when a network partition occurs (some nodes can't talk to each other), you can't guarantee both Consistency and Availability at the same time.

CAP is mainly about what happens when a network partition occurs.

Golden Rule

When there is NO partition, you can have all 3: Consistency, Availability and Partition Tolerance.

- **Key Concepts**

- ✓ Consistency
- ✓ Availability
- ✓ Partition Tolerance
- ✓ CAP Theorem
- ✓ Strong Consistency
- ✓ Eventual Consistency

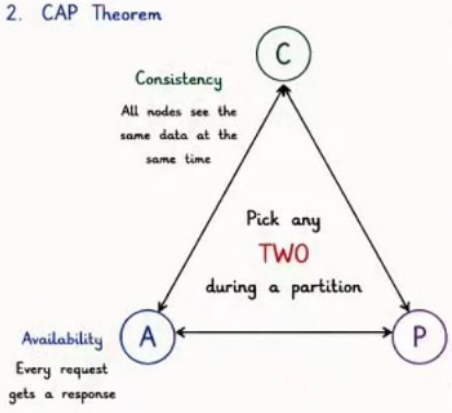
1. The 3 Guarantees

C Consistency (All nodes see the same data at the same time)
Every read returns the most recent write (or an error). No stale data.

A Availability (System always responds)
Every request receives a (non-error) response, even if some nodes are down.

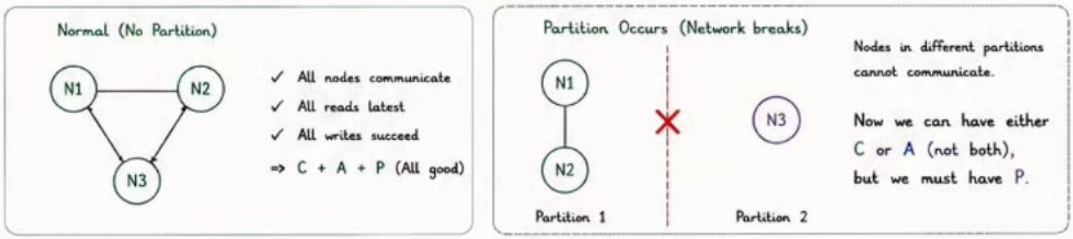
P Partition Tolerance (System keeps working despite network partitions)
The system continues to operate despite message loss or node disconnects.

2. CAP Theorem



⚠ When a network partition happens, you must choose between Consistency and Availability (you cannot have both), while you must have Partition Tolerance.

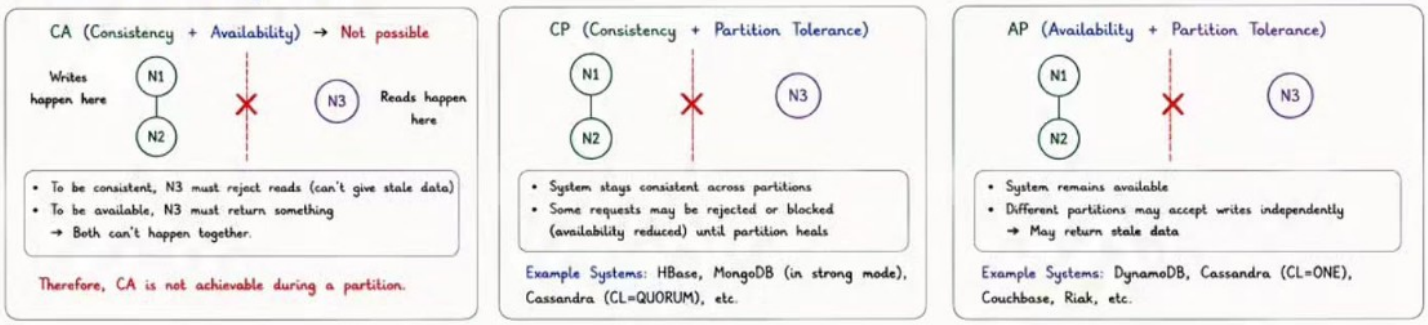
3. What happens during a NETWORK PARTITION?



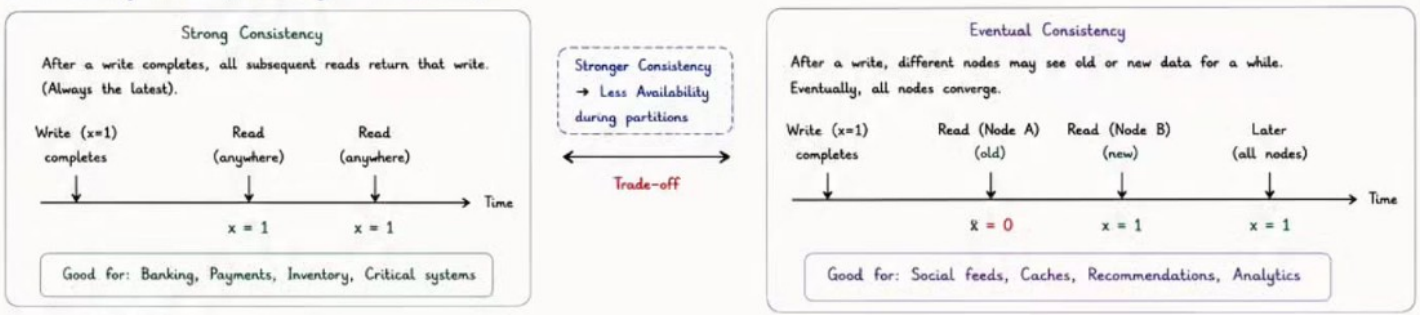
Important

CAP is not about systems in normal conditions. It is about the trade-offs you face during a network partition.

4. Possible Choices During a Partition



5. Consistency Models (How strong is "Consistent"?)



6. Real World Examples

System / Service	Consistency Model	CP or AP? (When Partition)	Why?
AWS DynamoDB	Eventual Consistency (default)	AP	High availability, global scale

Key Takeaways

✓ In a partition, you can choose Consistency or Availability

طوابير الرسائل

Message Queues

ماذا يشرح هذا القسم؟

تشرح الصورة وضع الأعمال في طابور بين المنتج والمستهلك، لفصل استقبال الطلب عن تنفيذه. يمكن للطابور امتصاص موجات الضغط ومعالجة الأعمال في الخلفية.

المحاور الأساسية

1. المنتج يرسل رسالة، والطابور يحتفظ بها، والمستهلك ينفذ العمل وفق سعة المعالجة.
2. إعادة المحاولة وطابور الرسائل الفاشلة والضغط العكسي تساعد إدارة الفشل والازدحام.
3. ضمانات التسليم تختلف؛ وقد تحتاج العملية إلى منع تكرار الأثر عند إعادة تسليم الرسالة.

مثال مبسط

يمكن إدراج مهمة إرسال بريد في الطابور بعد تنفيذ العملية الأساسية، ثم ينفذها عامل خلفي دون إبقاء المستخدم منتظرا.

الفكرة التي ينبغي تذكرها

حدد ضمان التسليم وسلوك إعادة المحاولة، ولا تفترض تنفيذ كل رسالة مرة واحدة تلقائيا.

12

MESSAGE QUEUES

SYSTEM DESIGN DECODED

Why Message Queues?

Har task immediately process karna zaroori nahi hota.
Queues help systems handle work asynchronously, reliably and at scale.

Decouple. Buffer. Smooth spikes. Increase reliability.

Golden Rule

Send the message. Trust the queue.
Process it when you can.
Don't block the main flow.

Key Concepts

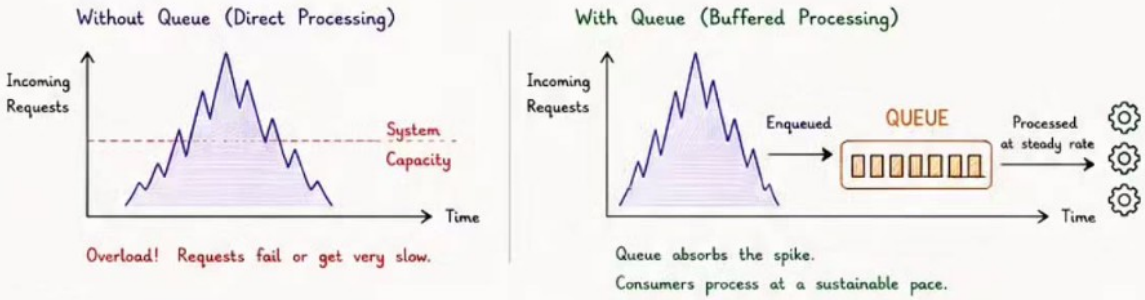
- ✓ Producer
- ✓ Queue
- ✓ Consumer
- ✓ Async Processing
- ✓ Buffering
- ✓ Retry
- ✓ Dead-letter Queue
- ✓ Backpressure

1. Basic Flow



- Producer doesn't wait
- Consumer pulls messages
- If consumer is slow, messages wait in queue
- This is async processing

2. Traffic Spike? Queue Absorbs It!



Benefit

- ✓ No message loss
- ✓ Smooth user experience
- ✓ Protect downstream systems
- ✓ Better resource utilization

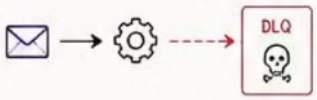
3. Important Patterns

Retry on Failure



- If processing fails (e.g., DB down), message is retried after some delay.
- Use exponential backoff to avoid retry storms.

Dead-Letter Queue (DLQ)



- If a message keeps failing even after multiple retries, move it to DLQ.
- Prevents poison messages from blocking the queue.
- DLQ can be inspected and reprocessed manually.

Backpressure



- When consumer is slow, queue fills up.
- Queue applies backpressure:
  - Reject new messages
  - Slow down producers
  - Or shed load gracefully

Buffering



- Queue acts as a buffer between systems with different speeds.
- Smoothes out traffic bursts.

4. Under the Hood

Durability
Messages are stored persistently until consumed.

Ordering
Messages are delivered in order (within a partition).

At-Least-Once
A message may be delivered more than once. Ensure idempotent consumers.

At-Most-Once
A message may get lost but never delivered twice.

Exactly-Once (Rare)
No duplicates, no loss. Hard to achieve end-to-end; use carefully.

5. Example Use Cases

Order Processing
Place order → enqueue email, payment, invoice generation.

Notifications
Send emails, SMS, push notifications reliably.

Video / Image Processing
Generate thumbnails, transcode videos.

Analytics Events
Collect events and process in batches.

Background Jobs
Reports, backups, data sync, ETL jobs.

6. Popular Implementations (Examples)

RabbitMQ · Apache Kafka · Amazon SQS · Google Pub/Sub · Redis Streams

7. Best Practices

- ✓ Keep messages small.
- ✓ Make consumers idempotent

النشر والاشتراك

Publish / Subscribe

ماذا يشرح هذا القسم؟

تقارن الصورة بين طابور توزع فيه الأعمال على المستهلكين، ونموذج نشر واشتراك ترسل فيه الأحداث إلى الجهات المشتركة في الموضوع. يساعد ذلك على فصل المنتج عن عدة أنظمة مهتمة بالحدث.

المحاور الأساسية

1. الناشر يعلن حدثا في موضوع، وتتابعه اشتراكات مستقلة.
2. التوزيع على عدة جهات يتيح التحليلات والإشعارات والفوترة دون استدعاء مباشر لكل منها.
3. الاختيار بين الطابور والنشر والاشتراك يعتمد على ما إذا كان العمل مطلوبا من جهة واحدة أو عدة جهات.

مثال مبسط

حدث إنشاء طلب قد تتابعه خدمة المخزون وخدمة الإشعارات ونظام التحليلات، ولكل جهة معالجتها المستقلة.

الفكرة التي ينبغي تذكرها

عرف الأحداث بوضوح، وميز توزيع العمل عن نشر الحدث إلى عدة مشتركين.

13

PUB / SUB (PUBLISH / SUBSCRIBE)

SYSTEM DESIGN DECODED

Why Pub/Sub?

Queue aur Pub/Sub dono messages exchange karte hain, but unka communication pattern alag hota hai.

If you have one worker → use Queue.
If you have many listeners → use Pub/Sub.

Golden Rule

Queues distribute work to workers.

Pub/Sub distributes events to listeners.

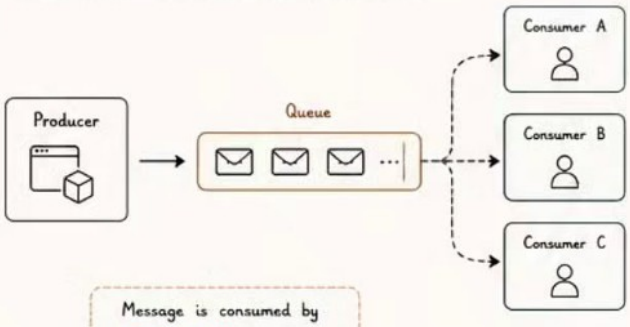
Use the right pattern for the right job.

Key Concepts

- ✓ Queue Model
- ✓ Publish / Subscribe Model
- ✓ Topic
- ✓ Multiple Consumers
- ✓ Event Fan-out
- ✓ Decoupling
- ✓ Loose Coupling
- ✓ Use Cases

1. Queue Model (Point-to-Point)

One message → One consumer (load balanced).



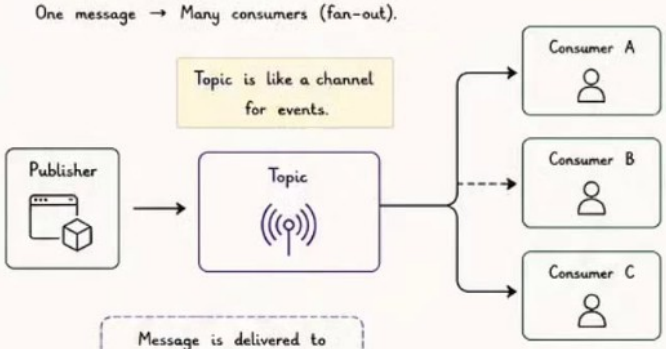
Message is consumed by ONLY ONE consumer.

Use Cases

- Background jobs
- Task processing
- Work distribution
- Email sending
- Image resizing

2. Publish / Subscribe Model

One message → Many consumers (fan-out).



Message is delivered to ALL interested consumers.

Use Cases

- Real-time notifications
- Activity feeds
- Analytics / Logging
- Cache invalidation
- IoT / Sensor events
- Audit trails

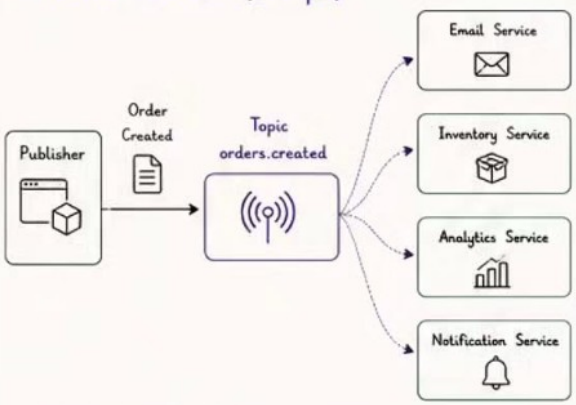
Benefits

- ✓ Loose coupling
- ✓ Easy to add new consumers
- ✓ Scales with many subscribers
- ✓ Great for event-driven systems

3. Queue vs Pub/Sub

Aspect	Queue (Point-to-Point)	Pub/Sub (Publish / Subscribe)
Communication	1 Producer → 1 Queue → 1 Consumer (One message → One consumer)	1 Publisher → 1 Topic → Many Consumers (One message → Many consumers)
Delivery	Competing consumers. Only one gets it.	All subscribers get a copy.
Message State	Message removed after consumption.	Message retained (as per retention policy). Delivered to all current subscribers.
Backlog Handling	Queue builds up if consumer is slow.	Each subscriber has its own position. They can catch up independently.
Ordering	Per queue ordering.	Per topic partition ordering (often eventual across subscribers).
Use For	Distributing work / tasks.	Distributing events / notifications.
Examples	RabbitMQ (Classic Queues), SQS (Standard)	Kafka, Google Pub/Sub, NATS, SNS

4. Event Fan-out (Example)



Same event → many different systems.
No tight coupling.

5. When to Use What?

Use Queue when...

- ✓ You have a task that needs to be processed by one worker.
- ✓ You want load balancing across workers.
- ✓ Order processing matters and only one

Use Pub/Sub when...

- ✓ You have an event that many systems care about.
- ✓ Systems are independent and can process in parallel.
- ✓ You want to add new consumers without affecting

6. Real World Examples

E-commerce	Order placed event → Inventory, Payment, Email, Analytics, Notification
Social Apps	New post → Feed service, Push service, Search indexing, Analytics
IoT / Devices	Sensor data → Monitoring, Alerts, Storage,

تحديد معدل الطلبات

Rate Limiting

ماذا يشرح هذا القسم؟

تشرح الصورة ضبط عدد الطلبات خلال فترة زمنية لحماية الموارد وتحقيق عدالة الاستخدام. وتقارن عدة خوارزميات تختلف في التعامل مع الاندفاعات وحدود النوافذ الزمنية.

المحاور الأساسية

1. النافذة الثابتة بسيطة، لكن الطلبات عند حدود نافذتين قد تسبب اندفاعا أكبر.
2. النافذة المنزلقة تراقب الاستخدام على فترة متحركة، مقابل كلفة تتبع أكبر.
3. دلو الرموز يسمح باندفاعات مضبوطة؛ ودلو التسريب ينظم معدل خروج العمل.

مثال مبسط

تعرض الصورة حدا لخمسة طلبات خلال عشر ثوان، وتميز الطلب المقبول عن الطلب المرفوض مع حالة كثرة الطلبات.

الفكرة التي ينبغي تذكرها

حدد نطاق الحد ومفتاحه: مستخدم أم عنوان شبكة أم عميل، واختر السياسة حسب الحاجة.

14

RATE LIMITING

SYSTEM DESIGN DECODED

Why Rate Limiting?

Protect the system and ensure fairness when clients send too many requests.

- Prevent abuse (DDoS, brute force)
- Protect downstream services
- Ensure fair usage
- Control costs

Golden Rule

Allow what is good.

Limit what is excessive.

Be fair to everyone.

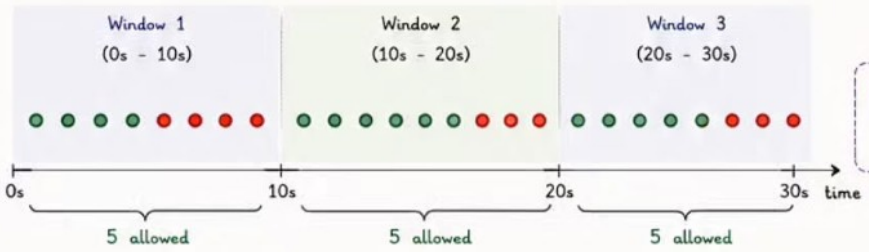
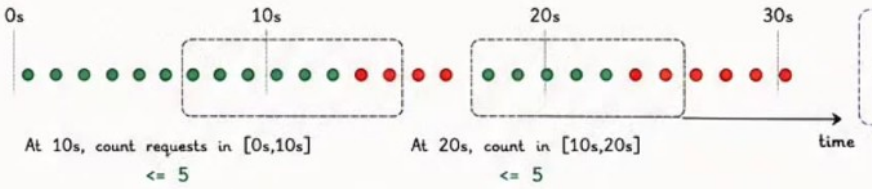
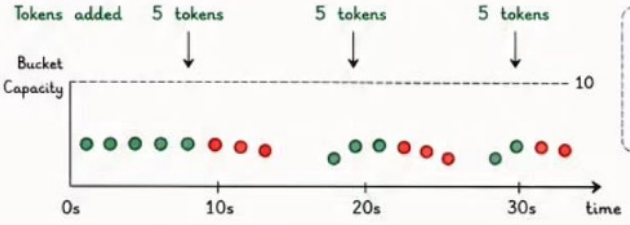
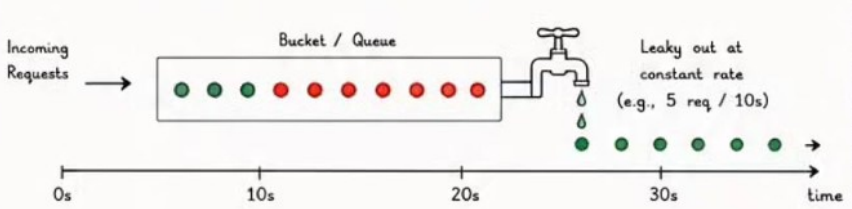
Key Concepts

- ✓ Requests per second (RPS)
- ✓ Burst vs Sustained traffic
- ✓ Rate limit per user / API key / IP
- ✓ Return 429 Too Many Requests
- ✓ Add Retry-After header

429 Too Many Requests

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 30
{
  "error": "rate limit exceeded",
  "message": "Try again later"
}
```

Let's say the limit is 5 requests per 10 seconds — Allowed — Blocked (429)

Algorithm	How it works	Request Timeline (each dot = 1 request)
1. Fixed Window	Time is divided into fixed windows. Limit is applied per window. Simple but can allow bursts at the edge of windows.	 Burst possible at window boundaries
2. Sliding Window	Look at the previous time window of size T (e.g., 10s). More accurate. Prevents boundary bursts.	 Smooth. No boundary burst.
3. Token Bucket	Tokens are added to the bucket at a constant rate. A request consumes 1 token. Allows bursts up to bucket capacity.	Example Configuration: Rate : 5 tokens / 10s; Capacity : 10 tokens; Cost : 1 token / request  Good balance of burst + sustained traffic.
4. Leaky Bucket	Requests enter a bucket (queue) and leak out at a constant rate. If bucket is full, new requests are blocked. Smooths traffic.	 Great for smoothing traffic. Higher latency possible.

Algorithm Comparison

Algorithm	Pros	Cons	Best Suited For
Fixed Window	Very simple, easy to implement	Boundary burst problem	Simple use cases, low accuracy needed
Sliding Window	More accurate, prevents bursts	Higher memory and computation	APIs, authentication, general purpose
Token Bucket	Allows bursts + steady rate	Need to tune rate and capacity	APIs, rate limits with burst tolerance

شبكة توزيع المحتوى وكاش الحافة

CDN & Edge Caching

ماذا يشرح هذا القسم؟

توضح الصورة تقديم نسخ المحتوى من مواقع قريبة من المستخدمين بدل العودة إلى المصدر البعيد في كل مرة. وتشمل الفكرة تخفيف الحمل على المصدر وتقليل زمن الوصول.

المحاور الأساسية

1. عند وجود نسخة صالحة على الحافة، يقدم الملف منها؛ وعند غيابها، يجلب من المصدر.
2. مدة الصلاحية وإبطال المحتوى وتغيير إصدار الملف تتحكم في تحديث النسخ الموزعة.
3. تختلف سياسة التخزين المؤقت للمحتوى العام عن المحتوى الخاص بالمستخدم.

مثال مبسط

يمكن أن يحصل مستخدم في منطقة بعيدة على صورة أو ملف تنسيق من نقطة حافة قريبة، بعد أن خزنت الشبكة نسخة منه.

الفكرة التي ينبغي تذكرها

اضبط ما يمكن تخزينه ومدة صلاحيته، وراع خصوصية المحتوى.

15

CDN & EDGE CACHING

SYSTEM DESIGN DECODED

Why not serve from origin every time?

Users are everywhere. Origin server is usually in one region.
CDN stores content at edge locations near users to reduce latency and load.

- Faster delivery
- Lower origin load
- Better availability
- Cost effective

Golden Rule

Cache static content at the edge.

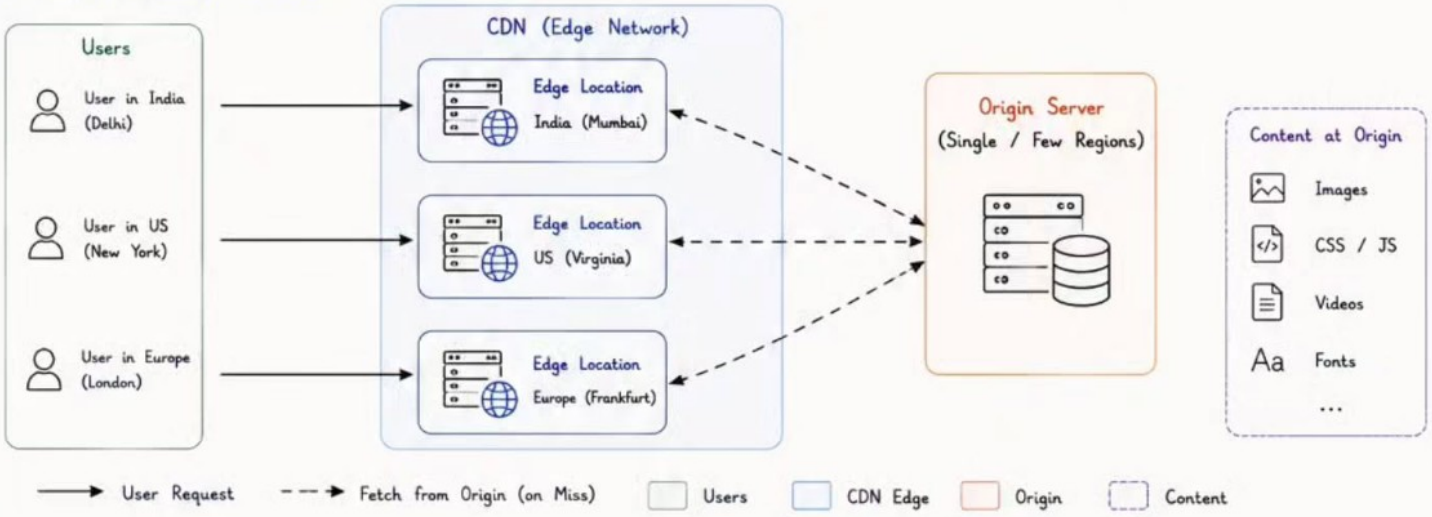
Go to origin only when you have to.

Key Concepts

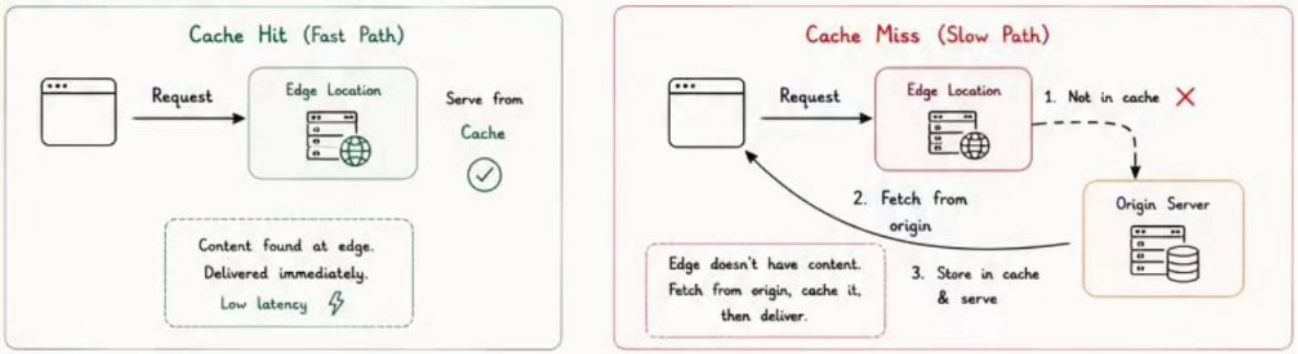
- ✓ CDN (Content Delivery Network)
- ✓ Edge Locations (PoPs)
- ✓ Origin Server
- ✓ Static Content (images, CSS, JS, fonts, videos)
- ✓ Cache Hit / Miss
- ✓ Latency Reduction
- ✓ Cache Invalidation / Purge

CDN brings content closer to users. Less distance, less latency.

1. CDN Architecture Overview



2. Cache Hit vs Miss

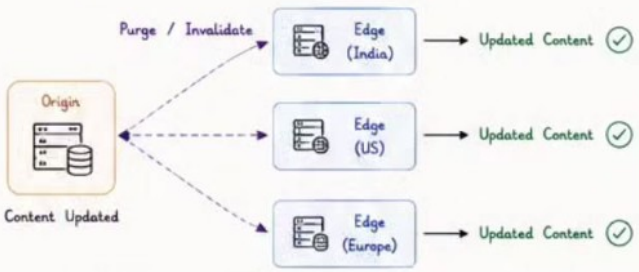


3. Cache Invalidation

When content changes at origin, we must invalidate or update caches, otherwise users see stale data.

Ways to Invalidate

- Purge by URL
- Purge by Path / Prefix
- Purge by Tag
- Versioned URLs (e.g. app.v2.js)
- Short TTL for frequently changing content



Good invalidation strategy = Fresh content + High cache hit ratio

4. Benefits

Lower Latency
Content served from nearest edge. Faster for users.

Reduced Origin Load
Fewer requests reach origin. Saves bandwidth & cost.

Better Scalability
Edge absorbs traffic spikes. Protects origin.

Better Availability
Content still available even if origin is down.

5. Best Practices

Key Takeaway

التخزين الموزع والملفات

Distributed Storage

ماذا يشرح هذا القسم؟

تشرح الصورة فصل الملف الفعلي عن بياناته الوصفية. يحتفظ التخزين المخصص بالصور والفيديوهات والملفات، بينما تحفظ قاعدة البيانات معلوماتها ومراجع الوصول إليها.

المحاور الأساسية

1. الرفع والتنزيل قد يستخدمان روابط موقعة مؤقتة، مع بقاء الصلاحيات تحت إدارة التطبيق.
2. الرفع متعدد الأجزاء يساعد التعامل مع الملفات الكبيرة واستئناف بعض العمليات.
3. التكرار والتوزيع وسياسات دورة حياة الملفات والحذف تؤثر في الموثوقية والكلفة.

مثال مبسط

يطلب التطبيق تصريح رفع، ثم يرفع العميل الملف إلى التخزين المخصص، وتحتفظ قاعدة البيانات باسم الملف وحجمه ومالكه ومفتاحه.

الفكرة التي ينبغي تذكرها

افصل الملفات عن بياناتها الوصفية، واضبط الصلاحيات وسياسات الاحتفاظ.

16 DISTRIBUTED STORAGE

SYSTEM DESIGN DECODED

Why not store large files in database?

Databases are great for structured data, not for massive files.

Storing files in DB leads to:

- ✗ Huge DB size & backups
- ✗ Poor performance
- ✗ High I/O on database
- ✗ Hard to scale

Use Object Storage

Built for durability, scalability, high availability and low cost.

Golden Rule

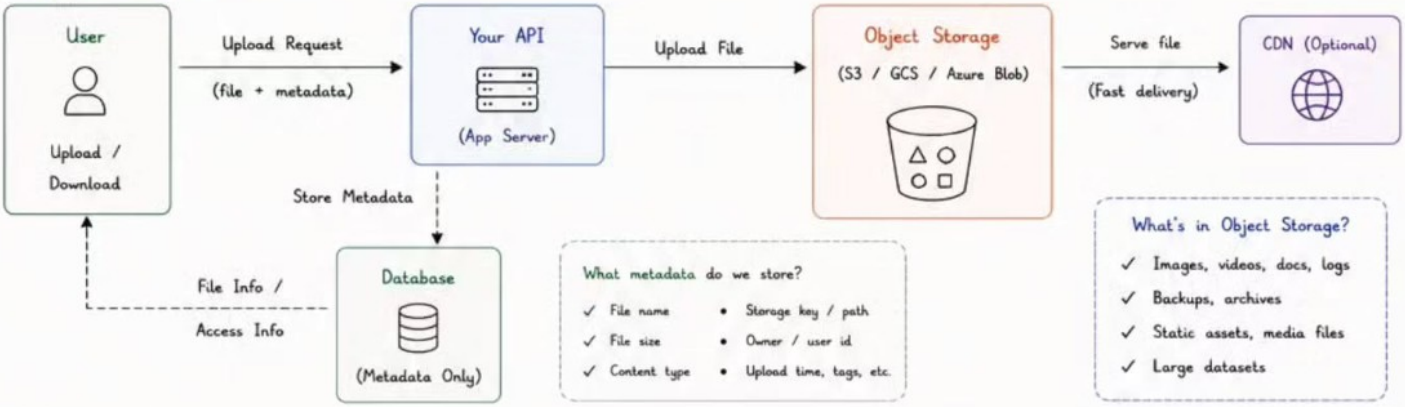
Store the file in Object Storage.

Store the reference (metadata) in Database.

Key Concepts

- ✓ Object / Blob Storage: Store files as objects (key + data)
- ✓ Metadata vs Actual File: DB stores info, not the file.
- ✓ Multipart Upload: Upload big files in parts.
- ✓ Pre-signed URLs: Secure, time-limited access.
- ✓ Replication: Data copied across multiple locations.
- ✓ Large-file Handling: Efficient upload, download & delivery.

1. High Level Architecture



2. Metadata vs Actual File

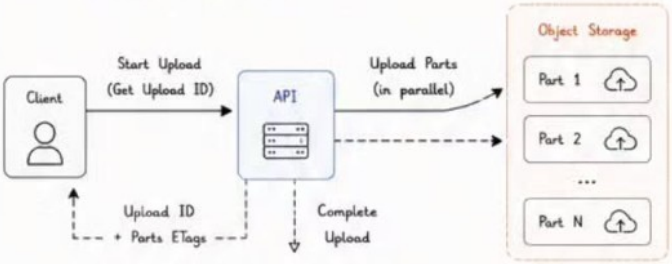
Database (Metadata Only)

id	file_name	file_size	content_type	storage_key	uploaded_by	created_at
1	resume.pdf	120 KB	application/pdf	resumes/1.pdf	user_101	2025-05-20
2	profile.jpg	2.4 MB	image/jpeg	profiles/101.jpg	user_101	2025-05-20
3	video.mp4	550 MB	video/mp4	videos/abc.mp4	user_202	2025-05-21
...	...	...	...	...	...	...

Object Storage (Actual Files)

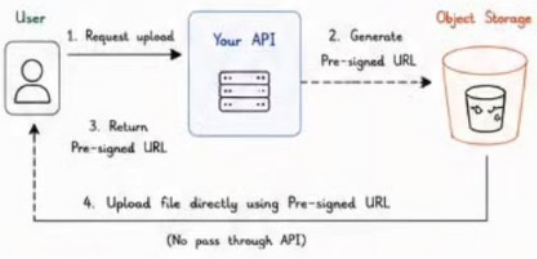
- resumes/1.pdf
- profiles/101.jpg
- videos/abc.mp4
- ...

3. Multipart Upload (for Large Files)



Benefits: Resume broken uploads, parallel upload, more reliable for big files.

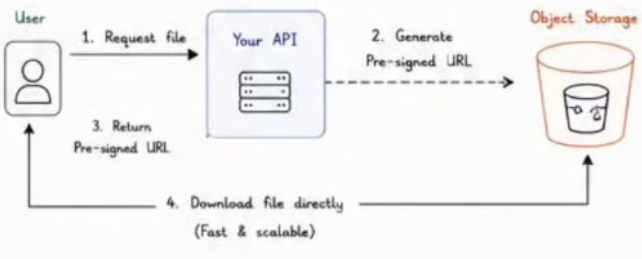
4. Pre-signed URLs (Secure Direct Access)



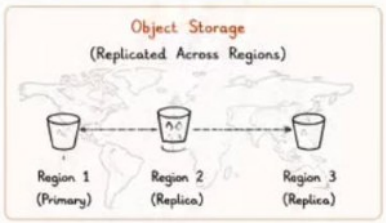
Why Pre-signed URLs?

- ✓ Offload bandwidth from your servers
- ✓ Faster uploads
- ✓ Secure (time-limited)
- ✓ Fine-grained access (upload/download only)

5. Download Flow (with Pre-signed URL)



6. Replication & Durability



- ✓ Files are replicated across multiple availability zones or regions.
- ✓ High durability (e.g. 99.999999999%)
- ✓ Survives disk, server, or even region failures.

7. Large-file Handling Best Practices

Use multipart upload for files > 100MB	Support resume for failed uploads	Compress files before upload (if possible)	Store metadata separately, not inside file	Set lifecycle policies (expire / archive old files)

توليد المعرفات الموزعة

Distributed ID Generation

ماذا يشرح هذا القسم؟

توضح الصورة الحاجة إلى معرفات فريدة عند إنشاء البيانات على عدة أجهزة. وتقارن الزيادة التلقائية والمعرفات العشوائية والمرتبة زمنيا وبنية سنوفليك.

المحاور الأساسية

1. المعرف المتزايد في قاعدة واحدة بسيط، لكنه يحتاج تنسيقا عند التوزيع بين أجهزة مستقلة.
2. تختلف البدائل في الطول والترتيب واحتمال التصادم وطريقة التوليد.
3. بنية سنوفليك المعروضة تجمع جزءا زمنيا ومعرف الجهاز وتسلسلا محليا.

مثال مبسط

قد تولد عدة خدمات طلبات في اللحظة نفسها؛ ويساعد معرف الجهاز مع التسلسل على تمييز المعرفات المولدة محليا.

الفكرة التي ينبغي تذكرها

اختر معرفا يلائم حاجتك إلى التفرد والترتيب والحجم، مع مراعاة الساعة والتنسيق.

17

DISTRIBUTED ID GENERATION

SYSTEM DESIGN DECODED

Why do we need it?

In distributed systems, multiple machines need to generate unique IDs without colliding, even at massive scale.

- Unique across space (machines)
- Roughly sortable (time-ordered)
- High throughput
- Works without central DB

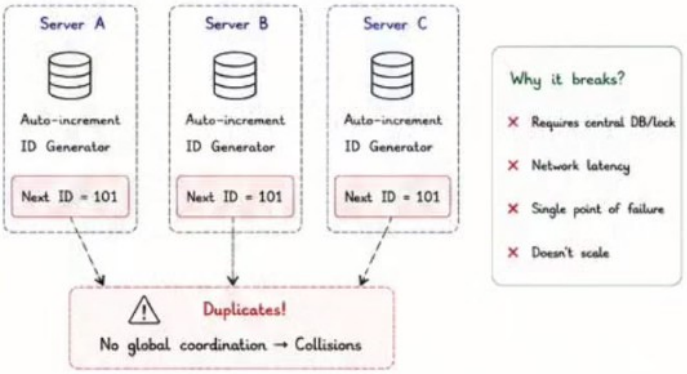
Golden Rule

IDs must be unique, scalable, and (ideally) ordered by time.

Key Concepts

- ✓ Auto-increment doesn't scale across machines
- ✓ IDs should be unique without coordination bottlenecks
- ✓ Time helps with ordering
- ✓ Machine (node) identity helps with uniqueness
- ✓ Sequence handles high throughput in the same millisecond
- ✓ Trade-offs: size, sortability, complexity

1. The Auto-increment Problem



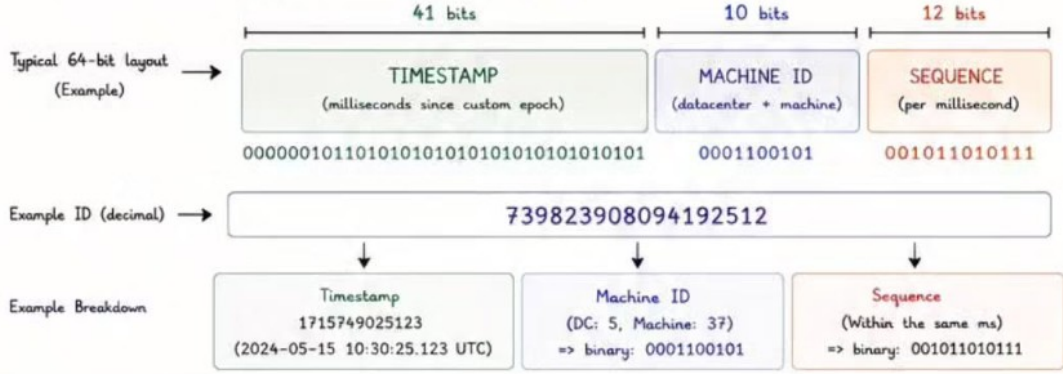
Why it breaks?

- ✗ Requires central DB/lock
- ✗ Network latency
- ✗ Single point of failure
- ✗ Doesn't scale

2. Common Approaches Overview

UUID (Universally Unique Identifier)	Random / timestamp-based 128-bit value	→	✓ Unique ✓ Not ordered
Timestamp-based IDs	Use time (ms/μs) + random/sequence	→	✓ Roughly ordered ✗ Possible collisions
Snowflake-style IDs	Time + Machine ID + Sequence	→	✓ Unique ✓ Ordered
Other variants	ULID, KSUID, CUID, NanoID, etc.		Different trade-offs for different needs

3. Snowflake-style ID Structure (Most popular in industry)



Why it works?

- ✓ Timestamp → ordering
- ✓ Machine ID → uniqueness across machines
- ✓ Sequence → handles many IDs in the same ms

Unique, ordered, fast!

4. Approaches Comparison

Approach	Example	Size	Uniqueness	Ordering	Throughput	Notes
Auto-increment (centralized)	1, 2, 3, 4, ...	Small	Global (with central DB)	Yes	Low-Medium (Bottleneck)	Doesn't work well in distributed setup
UUID (v4)	550e8400-e29b-41d4-a716-446655440000	128-bit	Very High	No	Very High	Random, not ordered, larger size
UUID (v7)	018f1e2a-9b7c-7f42-a716-446655440000	128-bit	Very High	Yes (time-ordered)	Very High	Newer UUID version with time ordering
Timestamp-based (+ random/seq)	1715749025123-8421	64-bit+	High (probabilistic)	Yes (roughly)	High	Simple to implement, collision possible
Snowflake-style	739823908094192512	64-bit	Very High	Yes	Very High	Industry standard (Scalable & efficient)

5. When to use what?

- Need simple & unique (no ordering needed)? → Use UUID v4
- Need time ordering & global uniqueness? → Use UUID v7 / ULID / KSUID

6. Best Practices

- ✓ Use a future custom epoch (e.g., Twitter Snowflake epoch).
- ✓ Monitor clock drift across machines (NTP).

تحمل الأعطال والموثوقية

Fault Tolerance & Reliability

ماذا يشرح هذا القسم؟

تشرح الصورة تصميم النظام على أساس توقع الأعطال، مع اكتشافها والتعافي منها وتقليل أثرها. الهدف أن يبقى فشل مكون واحد محدودا قدر الإمكان.

المحاور الأساسية

1. المهلات وإعادة المحاولة والتراجع التدريجي تمنع الانتظار غير المحدود وتخفف الازدحام.
2. قاطع الدائرة يحد الاستدعاءات المتكررة إلى خدمة فاشلة، والعزل يحمي بقية المكونات.
3. التكرار والتحويل إلى بديل وفحوص الصحة تدعم الاستمرارية، مع اختبار سيناريوهات الفشل.

مثال مبسط

عندما تتوقف خدمة، يمكن إيقاف تكرار الاتصال بها مؤقتا وتقديم نتيجة بديلة مناسبة بدل إبقاء كل الطلبات معلقة.

الفكرة التي ينبغي تذكرها

صمم لتدهور منضبط، ولا تجعل إعادة المحاولة تضاعف الضغط أو أثر العملية.

18 FAULT TOLERANCE & RELIABILITY

SYSTEM DESIGN DECODED

Failures happen.
Good systems are designed assuming they will.

Why it matters?

- Users expect the system to always work.
- Failures are normal, downtime is not.
- Reduces outage impact and data loss.
- Builds trust and reliability.

Golden Rule

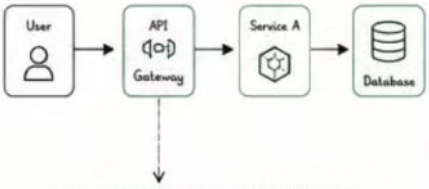
Design for failure.
Detect quickly.
Recover automatically.
Degrade gracefully.

Key Concepts

- ✓ Redundancy
- ✓ Timeouts
- ✓ Retries
- ✓ Exponential Backoff
- ✓ Circuit Breaker
- ✓ Failover
- ✓ Graceful Degradation
- ✓ Eliminate Single Point of Failure

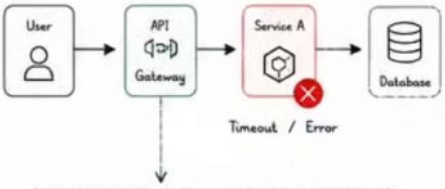
1. Request Flow: From Healthy → Failure → Fallback

A. Healthy Flow (Happy Path)



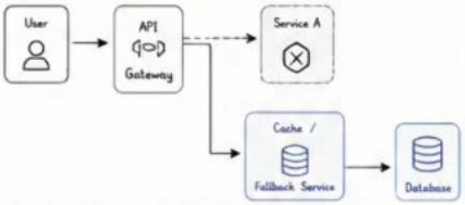
All services respond.
Request completes successfully.

B. Service Failure Occurs



Service A is down or not responding.
Requests start failing.

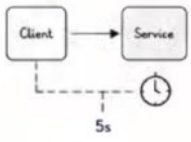
C. Fallback Path (System Stays Up)



Fallback service or cache serves the request.
User gets a response (maybe partial).
System remains available.

2. Core Resilience Patterns

1. Timeout



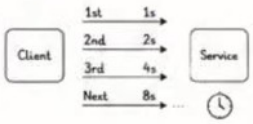
Don't wait forever.
Set a time limit and fail fast.
(e.g., 2s, 5s)

2. Retry



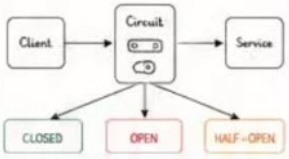
Retry failed requests.
Useful for transient failures.
(e.g., 3 retries)

3. Exponential Backoff



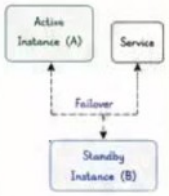
Wait longer between retries to avoid overwhelming the system.

4. Circuit Breaker



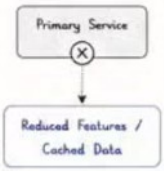
Stop sending requests to a failing service. Try again later.
Prevents cascading failures.

5. Failover



Automatic switch to a healthy instance when the active one fails.

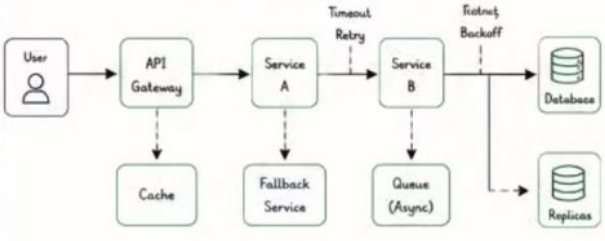
6. Graceful Degradation



If something fails, offer a limited but useful experience.
Better than complete failure.

3. Putting It Together

End-to-End Flow with Resilience



- ✓ If any service fails → retries → if still fails → fallback/cache/queue.
- ✓ If DB fails → failover to replica.
- ✓ Circuit breaker stops traffic to failing services.

4. Common Failure Types

- Network Failure — Packets lost, high latency, partition.
- Service Crash — Service down or process crash.
- Overload — Too many requests, resource exhausted.
- Slow Dependencies — DB slow, external API slow.

5. Quick Checklist

- ✓ Set timeouts for all calls.
- ✓ Use retries with exponential backoff.
- ✓ Implement circuit breaker.
- ✓ Keep services stateless when possible.
- ✓ Have multiple instances / replicas.
- ✓ Monitor, alert, and auto-heal.
- ✓ Test failure scenarios regularly.
- ✓ Design for graceful degradation.

قابلية الملاحظة والمراقبة

Observability

ماذا يشرح هذا القسم؟

توضح الصورة فهم ما يحدث داخل النظام باستخدام السجلات والمقاييس والتتبع. وتربط رحلة الطلب بمؤشرات زمن الاستجابة ومعدل الأخطاء وحجم المرور والتنبيهات.

المحاور الأساسية

1. السجلات تسجل الأحداث، والمقاييس تلخص سلوك النظام بالأرقام، والتتبع يربط مراحل الطلب بين الخدمات.
2. لوحات المراقبة والتنبيهات تساعد اكتشاف التأخر والفشل وتحديد المكون المرتبط بالمشكلة.
3. تحديد مؤشرات الخدمة وأهدافها يساعد التمييز بين عرض البيانات ومعرفة ما يتطلب إجراء.

مثال مبسط

إذا تأخر طلب، يكشف التتبع الخدمة التي استغرقت الزمن الأكبر، وتساعد السجلات والمقاييس في تفسير سبب التأخر.

الفكرة التي ينبغي تذكرها

اجمع المعلومات التي تساعد اتخاذ إجراء، واربط مراحل الطلب بمعرف تتبع واضح.

19

OBSERVABILITY

System slow hai — but kaunsa component slow hai?

SYSTEM DESIGN DECODED

Why Observability?

- Understand system behavior in real-time
- Detect issues quickly
- Debug faster
- Improve reliability & performance
- Make data-driven decisions

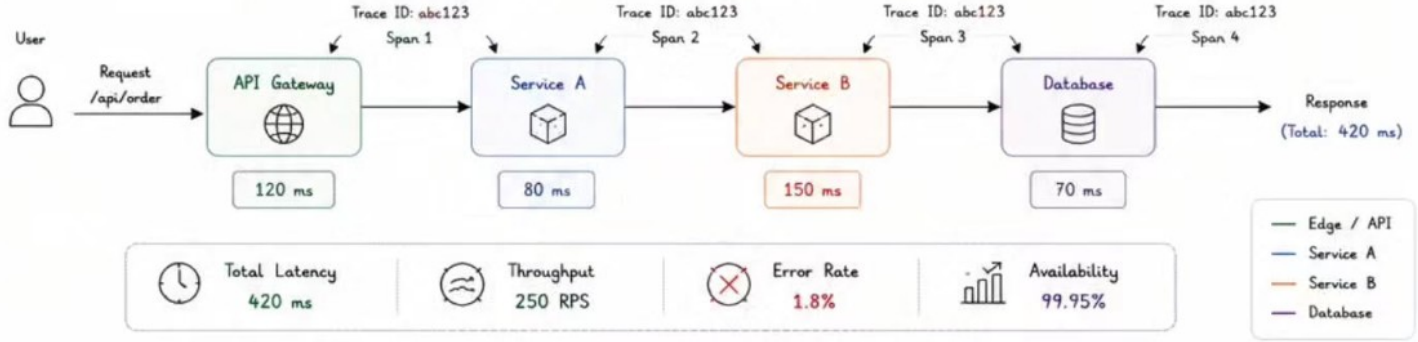
Golden Rule

If you can't measure it, you can't improve it. Instrument everything that matters.

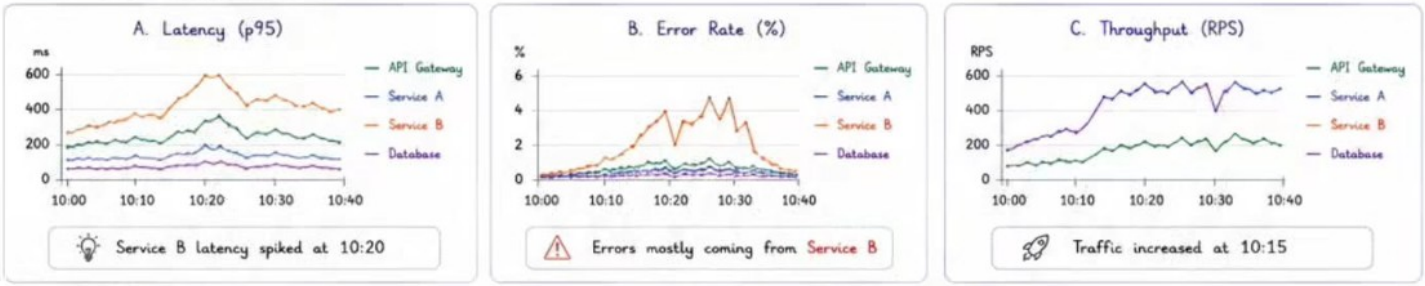
Key Concepts

- ✓ Logs (what happened?)
- ✓ Metrics (how much / how often?)
- ✓ Traces (how a request flowed?)
- ✓ Alerts (notify on bad signals)
- ✓ Latency (how slow?)
- ✓ Error Rate (%)
- ✓ Throughput (RPS / TPS)
- ✓ Distributed Tracing

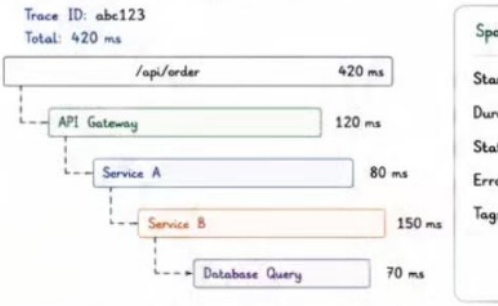
1. One Request Journey (End-to-End Observability)



2. What Observability Shows You



3. Distributed Tracing (Drill Down)



Span Details (Service B)

Start Time	10:20:15.123
Duration	150 ms
Status	OK
Error	-
Tags	orderId=9876 userId=42 region=IN

4. Alerts (Notify on Bad Signals)

Alert	Condition	Severity	Notification
High Latency	p95 latency > 500 ms for 5 min	High	PagerDuty / Slack
High Error Rate	Error rate > 2% for 5 min	High	PagerDuty / Slack
Low Throughput	RPS < 100 for 10 min	Medium	Email / Slack
Service Down	No successful health check	Critical	PagerDuty / Phone
High DB Latency	DB query p95 > 200 ms for 5 min	Medium	Email / Slack

5. Logs vs Metrics vs Traces

Aspect	Logs	Metrics	Traces
Purpose	Record discrete events	Measure numeric values over time	Track a request across services
Answers	What happened?	What is happening?	Why did it happen / where is it slow?
Granularity	High (individual events)	Aggregated (time-series)	Per-request (detailed)
Data Type	Unstructured / Semi-structured	Structured (numbers)	Structured (spans & relationships)
Examples	Error logs, access logs, audit logs	Latency, RPS, error rate, CPU, memory	Request flow, span duration, service dependencies
Best For	Debugging issues, root cause details	Monitoring health, alerting, dashboards	Finding bottlenecks, understanding dependencies
Storage	KB–MB per event	Lightweight	Moderate (sampled)

Good Observability =
The 3 pillars working together



الأمان ومفاضلات التصميم

Security & System Design Trade-offs

ماذا يشرح هذا القسم؟

تجمع الصورة أساسيات الأمان مع طريقة التفكير في الخيارات المعمارية. التصميم يتطلب موازنة متطلبات متعارضة أحيانا، وفق طبيعة المنتج والفريق والموارد.

المحاور الأساسية

1. أساسيات الأمان تشمل التحقق من الهوية والصلاحيات والتشفير والاتصال الآمن وحماية الأسرار والتحقق من المدخلات.
2. المفاضلات تشمل الاتساق والتوافر، وزمن الاستجابة والكلفة، والبساطة وقابلية التوسع.
3. ابدأ بفهم المشكلة، ثم صنف العائق وحدد البدائل وقارن أثرها بدل اختيار أداة لمجرد شيوعها.

مثال مبسط

قد تختار خدمة واحدة بسيطة في البداية، ثم تضيف كاشا أو نسخ قراءة أو فصلا للخدمات عند ظهور عائق محدد يمكن قياسه.

الفكرة التي ينبغي تذكرها

وثق سبب كل قرار وما يكسبه النظام وما يدفعه مقابله.

20 SECURITY & SYSTEM DESIGN TRADE-OFFS

SYSTEM DESIGN DECODED

Good systems are Secure, Scalable, Reliable and Balanced.

PART 1 — SECURITY BASICS

1. Authentication
Verify who the user is.
- Passwords
- MFA / OTP
- Tokens (JWT)

2. Authorization
Verify what the user can do.
- Roles & Permissions
- Access Control
- Least Privilege

3. Encryption
Protect data in transit and at rest.
- In-transit (TLS)
- At-rest (AES)
- End-to-end

4. HTTPS
Always use HTTPS to protect data in transit.
- TLS 1.2 / 1.3
- HSTS

5. Secrets
Keep secrets secret. Never hardcode.
- API Keys
- DB Passwords
- Use Secret Manager

6. Rate Limiting
Prevent abuse and protect your system.
- Requests per IP/User
- Burst + Sustained
- 429 Too Many Requests

7. Input Validation
Never trust user input.
- Validate & Sanitize
- Prevent SQLi, XSS
- Use allowlists

Security Golden Rule
Never trust, always verify. Protect data, limit access, and assume someone will try to break in.

PART 2 — ARCHITECTURE TRADE-OFFS

When you optimize for...	...you usually trade-off with...	What it means
Consistency	Availability	• Strong consistency can reduce availability during failures (CAP theorem). • Eventual consistency improves availability and partition tolerance.
Low Latency	Higher Cost	• Lower latency needs more resources, replicas, caches, edge locations. • Performance improvements usually cost more.
Simplicity	Scalability	• Simple systems are easy to build and maintain initially. • Highly scalable systems are more complex.
Performance	Maintainability	• Extreme performance often leads to complex code and infra. • Maintainable systems may sacrifice some raw performance.
Short-term Gain	Long-term Flexibility	• Quick hacks solve today's problem. • Good design keeps future options open.

There is no perfect architecture → only the right trade-offs for your context.

PART 3 — HOW TO THINK LIKE A SYSTEM DESIGNER (DECISION MAP)

1. Understand the Problem
- What are we building?
- Who are the users?
- What are the scale numbers?
- What are the requirements?
- What are the constraints?

2. Identify Key Concerns
- Scale (Traffic, Data, Growth)
- Performance (Latency, Throughput)
- Reliability (Failures, Availability)
- Consistency (Strong vs Eventual)
- Cost (Infra, Ops, Development)
- Security (Data, Access, Abuse)

Situation / Problem	Ask Yourself	Possible Approaches / Patterns
More Traffic / High Load	Can current system handle more requests?	Cache / Scale Horizontally / Load Balancer / CDN
More Data / Large Dataset	Can a single DB store and serve all data?	Replication / Sharding / Partitioning / Object Storage
Slow Processing / Long Tasks	Does user need to wait for the result?	Async Processing / Message Queue / Background Workers / Batching
More Failures / Unreliable System	How do we handle failures gracefully?	Retry (Backoff) / Failover / Circuit Breaker / Redundancy
High Latency / Slow Response	Where is the delay coming from?	CDN / Cache / Optimize Queries / Better Architecture
Security Risks / Abuse	How can we protect the system & data?	AuthN AuthZ / Rate Limiting / Input Validation / Encryption

3. Design, Iterate & Validate
Design → Review → Identify Bottlenecks → Optimize → Monitor

4. Monitor & Improve Continuously
Observe → Learn → Improve